

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Yield-Accruing Vaults	Documentation quality	High	
Timeline	2025-11-03 through 2025-11-07	Test quality	High	
Language	Solidity	Total Findings	13	Fixed: 10Acknowledged: 3
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	2	Fixed: 2
Specification	None	Medium severity findings ⓘ	3	Fixed: 3
Source Code	https://github.com/StartaleGroup/stablecoin-contracts  #dc6e7d1 	Low severity findings ⓘ	4	Fixed: 4
Auditors	- Rabib Islam Senior Auditing Engineer - Gereon Mendler Auditing Engineer - Andrei Stefan Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	4	Fixed: 1Acknowledged: 3

Summary of Findings

We have performed an audit focusing on Startale's `EarnVault` , `EarnVaultUpgradeable` , and `RewardRedistributor` contracts.

The system overall is designed to distribute yield to depositors in the `EarnVault` and the `SUSDSCVault` (the latter is out of scope). This is accomplished through the `RewardRedistributor` contract, which collects freshly minted USDSC yield, applies an optional fee, and then allocates the net yield to the two vaults based on a proportion comprised of user stake in each. It uses an integer carry system to mitigate rounding bias over successive distributions.

The `EarnVault` allows users to deposit USDSC and earn a direct share of the distributed yield, along with the potential for additional "boost" rewards in other ERC20 tokens. It uses an index-based accounting system to ensure yield is allocated proportionally to the principal deposits and the time they have been staked.

The audit revealed multiple bugs and vulnerabilities, the most impactful of which is a sandwich attack that allows siphoning of yield from honest participants via single-slot loans of USDSC. In addition, there are important accounting mistakes that must be fixed.

While the test suite is substantial, it ought to be augmented to account for additional edge cases demonstrated by exploits and accounting gaps discussed in this report.

Fix-Review Update 2025-11-20:

All of the issues have been either fixed or acknowledged, and the test suite has been substantially augmented.

ID	DESCRIPTION	SEVERITY	STATUS
STA-1	Leveraged TVL Manipulation via Sandwich Attack on <code>distribute()</code>	• High ⓘ	Fixed
STA-2	Boost Rewards Not Settled During Deposit Operations	• High ⓘ	Fixed
STA-3	Incorrect <code>S_base</code> Calculation with Donations Causes Yield Misallocation	• Medium ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
STA-4	<code>previewDistribute()</code> Does Not Perform an Exact Dry-Run of <code>distribute()</code>	• Medium ⓘ	Fixed
STA-5	Unexpected Boost Token Behavior Can Lead to Denial of Service	• Medium ⓘ	Fixed
STA-6	Unbounded <code>activeBoostTokens</code> Array Growth	• Low ⓘ	Fixed
STA-7	Boost Rewards Precision Loss without Carry Accounting	• Informational ⓘ	Acknowledged
STA-8	Ownership and Default Admin Role Can Be Renounced	• Low ⓘ	Fixed
STA-9	No Validation of M0 Yield Recipient Configuration	• Low ⓘ	Fixed
STA-10	Missing Validation for USDSC Address in <code>RewardRedistributor</code>	• Low ⓘ	Fixed
STA-11	Boost Rewards System Incompatible with Double-Entry Point Tokens	• Informational ⓘ	Acknowledged
STA-12	Lack of Support for Fee-on-Transfer and Rebasing Tokens	• Informational ⓘ	Acknowledged
STA-13	<code>depositWithPermit()</code> Cannot Be Used to Delegate Transaction Signing	• Informational ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.

- 3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
- 4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The scope focuses on the `RewardRedistributor` contract and `EarnVault` with its upgradeable variant.

Files Included

- `src/distributor/RewardRedistributor.sol`
- `src/intefaces/vaults/earn/*`
- `src/vaults/earn/EarnVaultUpgradeable.sol`
- `src/vaults/earn/EarnVaultStorageBase.sol`
- `src/vaults/earn/BoostRewardsLib.sol`
- `src/vaults/earn/EarnVault.sol`

Repo: `https://github.com/StartaleGroup/stablecoin-contracts`

Included Paths: `src/interfaces/vaults/earn/` , `src/vaults/earn/` , `src/distributor/RewardRedistributor.sol`

Extensions: `sol`

Files Excluded

- `src/coin/*`
- `src/external/*`
- `src/mock/*`
- `src/references/*`
- `test/*`

Operational Considerations

For the successful and secure operation of the `EarnVault` , `EarnVaultUpgradeable` , and `RewardRedistributor` contracts, a number of risks and considerations must be taken into account. These range from technical smart contract vulnerabilities to economic and operational risks.

EarnVault / EarnVaultUpgradeable

- Keeper Reliability: The distribution of both USDSC yield and boost rewards is dependent on external keepers (`yieldRedistributor` and `boostRewardKeeper`). If these keepers fail to call `onYield()` and `onBoostReward()` regularly, users will not see their rewards accrue in a timely manner, impacting user experience and trust in the platform.
- Centralized Control by Owner and Pauser: The Owner has significant control, including the ability to change critical addresses like the treasury and keepers. The Pauser role can halt all major functions of the contract. Malicious or compromised Owner or Pauser accounts could lead to a loss of funds or functionality. Using a multi-signature wallet or a time-lock for these roles is a common mitigation strategy.
- Yield Source Dependency: The yield distributed to `EarnVault` users is entirely dependent on the `RewardRedistributor` and the underlying USDSC minting mechanism. Any disruption to this source will directly impact the APY for vault users.
- Upgradeability Risks (`EarnVaultUpgradeable`): While upgradeability allows for bug fixes and new features, it also introduces significant risks.
 - Centralized Upgrade Power: The ability to upgrade the contract is a powerful privilege. If the key that controls upgrades is compromised, a malicious implementation could be deployed, potentially leading to a complete loss of user funds.
 - Storage Collisions: When upgrading, it's crucial to ensure that the storage layout of the new implementation is compatible with the old one to prevent data corruption.

RewardRedistributor

- Keeper (Operator) Necessity: The `distribute()` function must be called regularly by an address with the `OPERATOR_ROLE` . Failure to do so will cause yield to accumulate in the M0 extension without being distributed to the vaults, defeating the purpose of the ecosystem.
- Admin Privilege Concentration: The `DEFAULT_ADMIN_ROLE` has significant power to change core parameters, including the treasury address, the recipient vaults, and the fee structure. A compromised admin key could redirect all yield and fees to a malicious address. A multi-signature wallet is highly recommended for this role.
- Integration Failures: The `RewardRedistributor` is tightly coupled with the `EarnVault` , `sUSDSCVault` , and the USDSC token contract (which implements `IMYieldToOne`). An upgrade or change in any of these external contracts could break the `distribute` function if the interfaces change or if the external contracts start behaving unexpectedly.

Key Actors And Their Capabilities

EarnVault / EarnVaultUpgradeable

The `EarnVault` contract, in both its standard and upgradeable versions, defines a clear hierarchy of roles with specific privileges:

- Owner: The owner has the highest level of authority and is responsible for the overall administration of the contract. The privileges of the owner include:
 - Setting the addresses for the `yieldRedistributor`, `boostRewardKeeper`, treasury, and pauser.
 - Managing a blacklist of addresses that are restricted from interacting with the contract.
 - Executing emergency operations, such as sweeping surplus funds to the treasury when the contract is paused.

- Recovering accidentally sent ETH and other ERC20 tokens.
- YieldRedistributor: This role is designated to the RewardRedistributor contract. Its sole and critical function is to distribute USDSC yield to the vault's users by calling the onYield() function.
- BoostRewardKeeper: This address, typically a keeper or an automated operator, is authorized to distribute boost rewards (in the form of other ERC20 tokens) to the vault's users by calling the onBoostReward() function.
- Pauser: This role has the authority to pause and unpause the contract's operations. This is a crucial security feature that can be used to halt all activity in case of an emergency or a discovered vulnerability.
- Treasury: The treasury is the designated recipient of surplus funds that are swept from the contract during emergency operations.

RewardRedistributor

The RewardRedistributor contract manages the flow of yield from its source to the various distribution cohorts. It also has a well-defined set of roles and privileges:

- DEFAULT_ADMIN_ROLE : This role is granted to an admin address during the contract's deployment. The admin has the authority to manage the contract's parameters and to grant or revoke other roles. The specific privileges of this role include:
 - Updating the addresses of the treasury, earnVault, and susdscVault.
 - Updating the fee on the yield within the predefined MAX_FEE_BPS limit.
 - Pausing and unpausing the distribution of rewards.
- OPERATOR_ROLE : This role is granted to a keeper address, which is typically an automated system. The operator's primary responsibility is to call the distribute() function, which triggers the entire yield distribution process.

Findings

STA-1

Leveraged TVL Manipulation via Sandwich Attack on distribute()

• High ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: d65cfbdd53dee1f3f2005fa8728cab33f4af2453 .

The client provided the following explanation:

do think about: When lastSusdscTVL is 0 (no snapshot taken) or stale, preview functions will return misleading values. distribute validates snapshot age but preview doesn't.

File(s) affected: src/distributor/RewardRedistributor.sol , src/vaults/4626/SUSDSCVaultUpgradable.sol

Description: The RewardRedistributor's distribute() function reads the sUSDSC vault's totalAssets() at execution time to calculate yield splits. An attacker can build a leveraged position in the sUSDSC vault using a lending pool (e.g., Aave-style), sandwich the distribute() transaction, and capture disproportionate yield while stealing from EarnVault users. The attack uses borrowed USDSC as collateral in recursive loops to artificially inflate sUSDSC's TVL, manipulating the yield distribution formula.

```
// RewardRedistributor.sol:324 - Uses spot TVL (manipulable)
T_yield = susdscVault.totalAssets(); // ← Read at transaction execution time

// RewardRedistributor.sol:335-342 - Yield split calculation
if (useCarries) {
    uint256 numEarn = net * T_earn + _carryEarn;
    toEarn = numEarn / S_base; // EarnVault allocation

    uint256 numOn = net * T_yield + _carryOn;
    toOn = numOn / S_base; // sUSDSC allocation (inflated if T_yield manipulated)
}
```

A similar attack can be performed by depositing borrowed USDSC into the EarnVault .

Exploit Scenario:

Assuming sUSDSC vault has 10M USDSC, EarnVault has 5M principal, and distribute() is called every 3-4 hours with ~1M USDSC yield. Attacker uses 1M USDSC capital with access to a lending pool that accepts sUSDSC collateral at 90% LTV.

1. Attacker detects operator's distribute() transaction in mempool
2. Attacker frontruns with leverage loop (5 iterations): Deposit 1M USDSC → get 1M sUSDSC → borrow 900k USDSC. Deposit 900k → get 900k sUSDSC → borrow 810k. Continue until 4.095M total deposited using 3.685M borrowed + 1M original capital. This inflates sUSDSC vault from 10M to 14.095M USDSC.
3. Operator's distribute() executes with manipulated TVL:
 - Normal split would be: toEarn = 316,667 USDSC, toOn = 633,333 USDSC
 - Manipulated split: toEarn = 248,752 USDSC (lost 67,915), toOn = 701,248 USDSC (gained 67,915)
 - Attacker's 4.095M shares (29% of vault) capture 203,693 USDSC of the inflated yield
4. Attacker backruns: Redeem 4.095M shares for 4,298,693 USDSC, repay 3.685M loan (interest ~1.4 USDSC for 2 minutes), net profit 203,191 USDSC per attack

5. Attack repeatable 6 times daily = 1.2M USDSC profit/year (44,493% APY)

The vulnerability exists because ERC4626's share accounting is fair - attacker gets correct shares at deposit and correct assets at redemption. The exploit is in `distribute()` using spot TVL instead of time-delayed snapshot, allowing same-block manipulation.

Recommendation: Implement snapshot-based TVL mechanism to prevent same-block manipulation:

```
// In RewardRedistributor.sol
uint256 public lastSUSDCTVL;
uint256 public lastSnapshotBlock;

function snapshotTvl() external onlyRole(OPERATOR_ROLE) {
    lastSUSDCTVL = susdscVault.totalAssets();
    lastSnapshotBlock = block.number;
}

function distribute() external whenNotPaused onlyRole(OPERATOR_ROLE) nonReentrant {
    require(lastSnapshotBlock == block.number - 1, "Must snapshot in previous block");
    // Use lastSUSDCTVL instead of susdscVault.totalAssets() in calculations
    // ... rest of function
}
```

Operator workflow: Call `snapshotTvl()` in block N, then call `distribute()` in block N+1. This forces attacker to hold leveraged position for minimum 1 full block (~12 seconds), making attack 100x more expensive.

For defense-in-depth, add 1-hour withdrawal cooldown to sUSDSC vault and a minimum time post-deposit during which funds cannot be withdrawn.

STA-2 Boost Rewards Not Settled During Deposit Operations

• High ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `234dc385ca3a3eee66c7d792b4de0b5c0808e616`.

File(s) affected: `src/vaults/earn/EarnVaultUpgradeable.sol`, `src/vaults/earn/EarnVault.sol`, `src/vaults/earn/BoostRewardsLib.sol`

Description: The `deposit()` and `depositWithPermit()` functions settle USDSC yield via `_settle()` but do NOT settle boost rewards via `_settleBoost()`. This creates an accounting vulnerability where users can increase their principal without updating their boost reward indices, causing them to earn boost rewards on their new principal retroactively from their old index position. This allows users to steal boost rewards from other depositors.

```
// EarnVaultUpgradeable.sol:194-207 - deposit() function
function deposit(uint256 amount) external virtual whenNotPaused nonReentrant {
    EarnVaultStorage storage $ = _getStorage();
    _checkNotBlacklisted(msg.sender);
    if (amount == 0) revert IEarnVaultEventsAndErrors.ZeroAmount();

    _settle(msg.sender); // ← Only settles USDSC yield, NOT boost rewards!

    $.USDSC.safeTransferFrom(msg.sender, address(this), amount);
    $.principal[msg.sender] += amount; // ← Principal increased without settling boost
    $.totalPrincipal += amount;
    $.claimReserve += amount;

    emit Deposit(msg.sender, amount);
}
```

Compare with `withdraw()` which DOES settle boost rewards:

```
// EarnVaultUpgradeable.sol:249-272 - withdraw() function
function withdraw(uint256 amount) external virtual whenNotPaused nonReentrant {
    EarnVaultStorage storage $ = _getStorage();
    _checkNotBlacklisted(msg.sender);
    if (amount == 0) revert IEarnVaultEventsAndErrors.ZeroAmount();

    _settle(msg.sender); // Settles USDSC yield
```



```

uint256 p = $.principal[msg.sender];
if (amount > p) revert IEarnVaultEventsAndErrors.InsufficientPrincipal();

// Settle and claim ALL boost rewards in single loop
for (uint256 i = 0; i < $.activeBoostTokens.length; i++) {
    address token = $.activeBoostTokens[i];
    _settleBoost(msg.sender, token); // ← Settles boost rewards properly!
    BoostRewardsLib.claimBoostReward(/*...*/);
}
// ... rest of function
}

```

Boost reward indices track "yield per unit of principal" using the formula $\text{owed} = \text{principal} * (\text{boostGlobalIndex} - \text{userBoostIndex}) / \text{RAY}$. When a user deposits without settling boost rewards, their `userBoostIndex` remains at the old value while their `principal` increases, causing the calculation to apply the index delta to ALL their principal (old + new), not just the old principal that was actually deposited during that period.

Exploit Scenario:

Assuming boost rewards are distributed regularly:

1. Time T0 - Initial State:

- Alice deposits 1000 USDSC (first deposit)
- `principal[Alice] = 1000`
- `userBoostIndex[Alice][TokenA] = 0` (default for new user)
- `boostGlobalIndex[TokenA] = 0`

2. Time T1 - First Boost Distribution:

- Keeper distributes 1000 TokenA boost rewards to vault
- `boostGlobalIndex[TokenA]` increases: $0 + (1000 * 1e27) / 1000 = 1e27$
- Alice's pending rewards: $1000 * (1e27 - 0) / 1e27 = 1000$ TokenA ✓

3. Time T2 - Alice Deposits More WITHOUT Settlement:

- Alice calls `deposit(1000)` (doubles her position)
- `_settle()` is called → USDSC yield settled ✓
- `_settleBoost()` is NOT called → Boost indices NOT updated ✗
- Alice's state:
 - `principal[Alice] = 2000` (doubled!)
 - `userBoostIndex[Alice][TokenA] = 0` (STILL 0! Not updated to 1e27)
- Alice's 1000 TokenA is still pending in `userBoostAccrued`

4. Time T3 - Second Boost Distribution:

- Keeper distributes another 1000 TokenA
- Total distributed: 2000 TokenA in the system
- `boostGlobalIndex[TokenA]` increases: $1e27 + (1000 * 1e27) / 2000 = 1.5e27$

5. Time T4 - Alice Tries to Claim:

- Alice calls `claim()` or `withdraw()`
- `_settleBoost()` is called
- **Incorrect calculation:**
 - $\text{owed} = 2000 * (1.5e27 - 0) / 1e27 = 3000$ TokenA
- **Alice tries to claim 3000 TokenA when only 2000 exist!**
- Transaction REVERTS with `InsufficientBoostClaimReserve()`

6. Correct Calculation Should Be:

- Old principal (1000): earned T0→T1: $1000 * 1e27 / 1e27 = 1000$ TokenA
- Old principal (1000): earned T1→T3: $1000 * 0.5e27 / 1e27 = 500$ TokenA
- New principal (1000): earned T2→T3: $1000 * 0.5e27 / 1e27 = 500$ TokenA
- **Total correct: 2000 TokenA**

Recommendation: dd boost reward settlement to both deposit functions:

```

function deposit(uint256 amount) external virtual whenNotPaused nonReentrant {
    EarnVaultStorage storage $ = _getStorage();
    _checkNotBlacklisted(msg.sender);
    if (amount == 0) revert IEarnVaultEventsAndErrors.ZeroAmount();

    _settle(msg.sender);

    // ADD: Settle all boost rewards before changing principal
    for (uint256 i = 0; i < $.activeBoostTokens.length; i++) {
        _settleBoost(msg.sender, $.activeBoostTokens[i]);
    }

    $.USDSC.safeTransferFrom(msg.sender, address(this), amount);
    $.principal[msg.sender] += amount;
    $.totalPrincipal += amount;
    $.claimReserve += amount;
}

```

```

    emit Deposit(msg.sender, amount);
}

function depositWithPermit(
    uint256 amount,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external whenNotPaused nonReentrant {
    EarnVaultStorage storage $ = _getStorage();
    _checkNotBlacklisted(msg.sender);
    if (amount == 0) revert IEarnVaultEventsAndErrors.ZeroAmount();

    // Permit logic...
    try IERC20Permit(address($.USDSC)).permit(msg.sender, address(this), amount, deadline, v, r, s) {
    } catch {
        revert IEarnVaultEventsAndErrors.PermittedFailed();
    }

    _settle(msg.sender);

    // ADD: Settle all boost rewards before changing principal
    for (uint256 i = 0; i < $.activeBoostTokens.length; i++) {
        _settleBoost(msg.sender, $.activeBoostTokens[i]);
    }

    $.USDSC.safeTransferFrom(msg.sender, address(this), amount);
    $.principal[msg.sender] += amount;
    $.totalPrincipal += amount;
    $.claimReserve += amount;

    emit Deposit(msg.sender, amount);
}

```

STA-3

Incorrect S_base Calculation with Donations Causes Yield Misallocation

• Medium ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 1d9925822deba10eca8aad8a51b5021284fb530 .

File(s) affected: src/distributor/RewardRedistributor.sol

Description: The RewardRedistributor's `distribute()` function contains an accounting bug when tokens are donated to the contract. The function includes `balanceBefore` (donations) in the `gross` amount passed to `_calculateSplit()`, which causes `S_base` (supply base) to be artificially reduced. This results in vaults receiving a higher percentage of yield than intended, with the treasury receiving correspondingly less. **Note: A PoC was created to test for arithmetic underflow/freeze scenario, but testing confirmed this does NOT cause protocol freeze - instead it causes accounting imbalance favoring vault users over the protocol treasury.**

```

// RewardRedistributor.sol:151-165 - The root cause
function distribute() external whenNotPaused onlyRole(OPERATOR_ROLE) nonReentrant {
    uint256 balanceBefore = IERC20(USDSC_ADDRESS).balanceOf(address(this)); // Includes donations!
    uint256 minted = IMYieldToOne(USDSC_ADDRESS).claimYield(); // Only M0 yield
    uint256 gross = balanceBefore + minted; // Total including donations

    // Passes GROSS (not just minted) to _calculateSplit
    (feeToStartale, toEarn, toOn, toStartaleExtra, S_base, T_earn, T_yield) = _calculateSplit(gross,
true, false);
    ^^^^^^

}

// RewardRedistributor.sol:316-320 - S_base calculation
uint256 SNow = IERC20(USDSC_ADDRESS).totalSupply();
if (preMint) {

```

```

    S_base = SNow;
  } else {
    S_base = SNow > minted ? SNow - minted : 0; // Subtracts GROSS, not just actual minted!
  }

// RewardRedistributor.sol:345 - Underflow point
toStartaleExtra = net - (toEarn + toOn); // Reverts when toEarn + toOn > net

```

Exploit Scenario:

Assuming normal protocol state with USDSC totalSupply = 6M, T_earn (EarnVault principal) = 2M, T_yield (sUSDSC vault assets) = 3M, and actual M0 yield = 50k:

1. User accidentally transfers 1M USDSC directly to RewardRedistributor contract (wrong address or protocol error)
2. Operator calls `distribute()`: `balanceBefore` = 1M, `minted` = 50k (increases totalSupply to 6.05M), `gross` = 1.05M passed to `_calculateSplit`
3. In `_calculateSplit`: `feeToStartale` = 105k, `net` = 945k, `SNow` = 6.05M, `S_base` = 6.05M - 1.05M = 5M (WRONG! Should be 6M), `T_earn` = 2M, `T_yield` = 3M
4. Allocation calculation: `toEarn` = 378k (should be 315k), `toOn` = 567k (should be 472.5k), `toStartaleExtra` = 0 (should be 157.5k)
5. Vaults receive 100% of net yield, treasury receives 0 remainder, losing ~157k USDSC per distribution

A comprehensive PoC was created (`test/POC_ARITHMETIC_UNDERFLOW_TEST.sol`) that initially suspected arithmetic underflow causing protocol freeze, but testing confirmed NO underflow occurs - allocations exactly match available funds. The actual bug is `S_base` being artificially LOW, causing over-allocation to vaults.

Recommendation: Remove `balanceBefore` logic and only distribute actual minted yield to ensure `S_base` is correctly calculated:

```

function distribute() external whenNotPaused onlyRole(OPERATOR_ROLE) nonReentrant {
    uint256 minted = IMYieldToOne(USDSC_ADDRESS).claimYield();
    if (minted == 0) return;
    (feeToStartale, toEarn, toOn, toStartaleExtra, S_base, T_earn, T_yield) = _calculateSplit(minted,
    true, false);
    // ... rest of existing logic unchanged
}

// Add separate function to recover donations
function recoverDonations() external onlyRole(DEFAULT_ADMIN_ROLE) nonReentrant {
    uint256 balance = IERC20(USDSC_ADDRESS).balanceOf(address(this));
    if (balance > 0) {
        IERC20(USDSC_ADDRESS).safeTransfer(treasury, balance);
        emit DonationsRecovered(balance);
    }
}

```

STA-4

• Medium ⓘ Fixed

`previewDistribute()` Does Not Perform an Exact Dry-Run of `distribute()`

✓ Update

Marked as "Fixed" by the client.

Addressed in: `d65cfbdd53dee1f3f2005fa8728cab33f4af2453` .

The client provided the following explanation:

`STA-1` and `STA-4` are fixed in same PRs / final merge commit because they are highly related.

File(s) affected: `RewardRedistributor.sol`

Description: The NatSpec documentation of `previewDistribute()` describes the function as the following:

Exact dry-run of {distribute} against current chain **state** (includes carries).

However, `distribute()` includes both yield newly minted in the function call together with the balance of the contract prior to the minting in the calculation of the distributed USDSC. In contrast, `previewDistribute()` only includes the (hypothetical) newly minted yield. Hence, the latter function does not in fact perform an exact dry-run of the `distribute()` function, leading to inaccurate estimates when the contract already contains some USDSC.

Recommendation: Correct the implementation by properly incorporating the extant USDSC balance in the distribution amount, such that `distribute()` is properly simulated. Consider adding a new, differently-named function that preserves the current `previewDistribute()`

functionality.

STA-5

Unexpected Boost Token Behavior Can Lead to Denial of Service

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `5bfe651bac6ce75382929cb32c6a342a9e20921c` .

File(s) affected: `BoostRewardsLib.sol` , `EarnVault.sol` , `EarnVaultUpgradeable.sol`

Description: On calls of `distributeBoostReward()` and `claimBoostReward()` , calls are also made to the `transfer()` functions of all token contracts contained in the `activeBoostTokens` array. These calls occur in `EarnVault` functions: `withdraw()` , `claim()` , and `onBoostReward()` . However, considering these tokens are contracts external to the Startale project, they may exhibited unexpected behavior. For instance, if funds are frozen, they may revert on transfer. This would cause all of the above `EarnVault` functions to revert. However, importantly, once a token is added to `activeBoostTokens` , it cannot be removed. One token having its funds frozen would result in a denial-of-service that would prevent any funds from leaving the `EarnVault` .

Recommendation: Allow the removal of tokens from the `activeBoostTokens` array. Additionally, allow users to claim or withdraw specific tokens, to the exclusion of others.

STA-6 Unbounded `activeBoostTokens` Array Growth

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `fadd42341a39a93c38d5f7fe5a6f0c4ba74c62df` .

File(s) affected: `src/vaults/earn/BoostRewardsLib.sol` , `src/vaults/earn/EarnVaultUpgradeable.sol` , `src/vaults/earn/EarnVault.sol`

Description: The `activeBoostTokens` array grows every time a new boost token is distributed and never shrinks. The array is iterated in every `withdraw()` and `claim()` operation to settle and claim all boost rewards. As the array grows, gas costs for user operations increase linearly without bound.

In extreme cases, this can lead to denial of service for functions that loop over the array, if the gas required to call these functions exceeds block/transaction gas limit gaps.

```
// BoostRewardsLib.sol:67-72
// Track active boost tokens (only add if not already tracked)
if (boostTokenIndex[token] == 0) {
    // Token not tracked yet, add to array and set index
    activeBoostTokens.push(token);
    boostTokenIndex[token] = activeBoostTokens.length; // 1-based index
}
```

```
// EarnVaultUpgradeable.sol:260-272 (in withdraw())
// Settle and claim ALL boost rewards in single loop
for (uint256 i = 0; i < $.activeBoostTokens.length; i++) {
    address token = $.activeBoostTokens[i];
    _settleBoost(msg.sender, token);
    BoostRewardsLib.claimBoostReward(
        msg.sender,
        token,
        p, // Use original principal before withdrawal
        $.userBoostIndex[msg.sender][token],
        $.boostGlobalIndex[token],
        $.userBoostAccrued,
        $.boostClaimReserve
    );
}
```

```
// EarnVaultUpgradeable.sol:317-332 (in claim())
// Settle and claim all boost rewards in single loop
```

```

for (uint256 i = 0; i < $.activeBoostTokens.length; i++) {
    address token = $.activeBoostTokens[i];
    _settleBoost(msg.sender, token);
    uint256 claimedAmount = BoostRewardsLib.claimBoostReward(
        msg.sender,
        token,
        $.principal[msg.sender],
        $.userBoostIndex[msg.sender][token],
        $.boostGlobalIndex[token],
        $.userBoostAccrued,
        $.boostClaimReserve
    );
    if (claimedAmount > 0) {
        hasBoostClaim = true;
    }
}

```

Exploit Scenario:

Protocol distributes boost rewards in 50 different tokens over 2 years (ASTR, DOT, GLMR, BNC, etc.). With `activeBoostTokens.length = 50`, each `withdraw()` or `claim()` loops through all 50 tokens. Each loop iteration requires 2 SLOAD for indices, SLOAD for accrued, and potential SSTORE for settlement (5,000 gas per $\rightarrow$ ken). Total extra gas for 50 $\rightarrow$ 250,000 gas just for boost settling. As the array grows to 100+ tokens, user transactions may hit the block gas limit, preventing users from withdrawing or claiming their funds due to gas exhaustion. All users pay gas costs for all historical boost tokens even if they never claimed those rewards, with no mechanism to remove old/inactive tokens.

Recommendation: Implement a token pruning mechanism to remove inactive tokens and optionally add a maximum array size limit:

```

function pruneInactiveBoostToken(address token) external onlyOwner {
    require(boostClaimReserve[token] == 0, "Token still has claimable rewards");
    uint256 index = boostTokenIndex[token] - 1;
    uint256 lastIndex = activeBoostTokens.length - 1;
    if (index != lastIndex) {
        address lastToken = activeBoostTokens[lastIndex];
        activeBoostTokens[index] = lastToken;
        boostTokenIndex[lastToken] = index + 1;
    }
    activeBoostTokens.pop();
    delete boostTokenIndex[token];
}

// Optional: Add maximum limit
uint256 public constant MAX_BOOST_TOKENS = 20;

```

STA-7

Boost Rewards Precision Loss without Carry Accounting

• Informational ⓘ

Acknowledged

Update

Marked as "Acknowledged" by the client.

File(s) affected: `src/vaults/earn/BoostRewardsLib.sol`

Description: Boost reward distribution does not use carry accounting like USDSC yield does. When boost rewards are distributed, the remainder from integer division is permanently lost rather than being tracked in a carry accumulator. Over time and many distributions, users lose small amounts of boost rewards that accumulate in the protocol.

```

// BoostRewardsLib.sol:54-62
// Deposits exist: distribute proportionally based on principal (same as USDSC yield)
if (bal < boostClaimReserve[token] + amount) revert
IEarnVaultEventsAndErrors.InsufficientBoostClaimReserve();

// Update boost global index for this token (same logic as USDSC yield)
uint256 carryRay = 0; // We use 0 for boost rewards as precision loss is negligible
unchecked {
    uint256 num = amount * RAY + carryRay; // carryRay always 0
    uint256 delta = num / totalPrincipal;
}

```

```
    boostGlobalIndex[token] += delta;
}
```

Compare to USDSC yield which DOES use carry:

```
// EarnVaultUpgradeable.sol:362-367
unchecked {
    uint256 num = amount * $.RAY + $_carryRay; // Uses stored carry
    uint256 delta = num / $.totalPrincipal;
    $_carryRay = num % $.totalPrincipal; // Stores new carry
    $.globalIndex += delta;
}
```

Exploit Scenario:

With non-round `totalPrincipal` values (e.g., 1,234,567 USDSC), each boost distribution loses precision. For example, distributing 100 tokens: $\text{num} = 100\text{e}18 * 1\text{e}27 = 1\text{e}47$, $\text{delta} = 1\text{e}47 / 1.234567\text{e}24 \approx 8.1\text{e}22$, with remainder $\approx 0.1\text{e}45$ permanently lost without carry tracking. Over 1000 distributions with average 0.01% precision loss per distribution, total loss reaches ~10% of total boost rewards. These lost tokens remain in the contract, benefiting the protocol at user expense.

Recommendation: Add per-token carry storage and implement carry accounting for boost rewards:

```
// In EarnVaultStorageBase.sol - add to storage struct
mapping(address => uint256) boostCarryRay;

// In BoostRewardsLib.sol - update distributeBoostReward function
uint256 _carryRay = boostCarryRay[token];
unchecked {
    uint256 num = amount * RAY + _carryRay;
    uint256 delta = num / totalPrincipal;
    boostCarryRay[token] = num % totalPrincipal; // Store carry remainder
    boostGlobalIndex[token] += delta;
}
```

STA-8 Ownership and Default Admin Role Can Be Renounced

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `fb1ea3218c1d5a9b25d7e330e4f1432149371d4d`, `230d733495237e6c8688340bec5f5daa2128ff2c`.

File(s) affected: `RewardRedistributor.sol`, `EarnVault.sol`, `EarnVaultUpgradeable.sol`

Description: `RewardRedistributor` inherits OpenZeppelin's `AccessControl` contract. The latter contract endows the `DEFAULT_ADMIN_ROLE` with wide-ranging privileges over determining the provision of roles in the contract. However, it is possible for owners of the role to renounce their role using the function `renounceRole()`, thereby leaving the contract without any top-level admin.

`EarnVault` and `EarnVaultUpgradeable` inherit OpenZeppelin's `Ownable2Step` and `Ownable2StepUpgradeable` respectively. Similar to the above case, the contracts contain an `owner` address that has access to a number of important functions, and they also can call `renounceOwnership()` to set the `owner` address to `address(0)`.

Recommendation: In order to avoid loss of admin privileges, consider the following:

1. Override `_revokeRole()` such that it reverts when the role is `DEFAULT_ADMIN_ROLE` and there is only one address for it remaining;
2. Override `renounceOwnership()` to revert.

STA-9 No Validation of M0 Yield Recipient Configuration

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `25c7608009f015b6954f994d46ab06f7c144c089`.

File(s) affected: `src/distributor/RewardRedistributor.sol`,

Description: The `RewardRedistributor` depends on being the `yieldRecipient` for the USDSC (MYieldToOne) contract to receive yield from M0. However, there is no validation that confirms the `RewardRedistributor` is actually set as the yield recipient. The M0 protocol allows the

`YIELD_RECIPIENT_MANAGER_ROLE` to change the yield recipient at any time via `setYieldRecipient()`, which would redirect all yield to a different address without the RewardRedistributor being aware.

```
// RewardRedistributor.sol:150-153
function distribute() external whenNotPaused onlyRole(OPERATOR_ROLE) nonReentrant {
    uint256 balanceBefore = IERC20(USDSC_ADDRESS).balanceOf(address(this));
    uint256 minted = IMYieldToOne(USDSC_ADDRESS).claimYield(); // ← No validation
    uint256 gross = balanceBefore + minted;
```

```
// IMYieldToOne interface (M0 protocol)
function setYieldRecipient(address yieldRecipient) external; // Callable by YIELD_RECIPIENT_MANAGER_ROLE
function yieldRecipient() external view returns (address);
```

Exploit Scenario:

RewardRedistributor is deployed and correctly set as USDSC yield recipient. M0's `YIELD_RECIPIENT_MANAGER_ROLE` calls `setYieldRecipient(newAddress)` due to governance decision, compromised keys, or compliance requirement. All future yield from USDSC goes to `newAddress` instead of RewardRedistributor. When operator calls `distribute()`, `claimYield()` succeeds but mints tokens to the new recipient, with RewardRedistributor receiving `minted = 0`. No yield distributed to EarnVault or sUSDSC users - protocol silently fails with no revert. Users' funds remain safe but earn zero yield indefinitely while actual yield goes elsewhere. The protocol has a critical external dependency with no validation. Unlike blacklisting (which reverts), a yield recipient change succeeds silently with `minted = 0`, causing complete loss of yield for all users while principal remains withdrawable.

Recommendation: Add validation check to ensure RewardRedistributor is still the yield recipient:

```
function distribute() external whenNotPaused onlyRole(OPERATOR_ROLE) nonReentrant {
    address currentRecipient = IMYieldToOne(USDSC_ADDRESS).yieldRecipient();
    if (currentRecipient != address(this)) {
        revert IRewardRedistributorEventsAndErrors.YieldRecipientChanged(currentRecipient);
    }
    // ... rest of function
}
```

STA-10 Missing Validation for USDSC Address in RewardRedistributor

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 2697cd8c5dff4a1320cebfaace476de000524e16 .

File(s) affected: `src/distributor/RewardRedistributor.sol`

Description: The RewardRedistributor constructor sets `USDSC_ADDRESS` as immutable but doesn't validate that this address actually implements the required interfaces (IERC20 and IMYieldToOne). If an incorrect address is provided during deployment, the contract will be permanently broken.

```
// RewardRedistributor.sol:67-82
constructor(address usdscAddress, address treasuryAddr, IEarnVault earnV, IERC4626 sVault, address admin,
address keeper) {
    if (usdscAddress == address(0)) revert
    IRewardRedistributorEventsAndErrors.ZeroAddress('USDSC_ADDRESS');
    if (treasuryAddr == address(0)) revert IRewardRedistributorEventsAndErrors.ZeroAddress('treasury');
    if (address(earnV) == address(0)) revert
    IRewardRedistributorEventsAndErrors.ZeroAddress('earnVault');
    if (address(sVault) == address(0)) revert
    IRewardRedistributorEventsAndErrors.ZeroAddress('susdscVault');
    if (admin == address(0)) revert IRewardRedistributorEventsAndErrors.ZeroAddress('admin');
    if (keeper == address(0)) revert IRewardRedistributorEventsAndErrors.ZeroAddress('keeper');

    USDSC_ADDRESS = usdscAddress; // No interface validation
    // ...
}
```

Exploit Scenario:

If wrong address is provided during deployment, contract must be redeployed as there's no way to update immutable `USDSC_ADDRESS`. Could

cause DoS on first `distribute()` call when interfaces are invoked. Low severity as it's typically caught during deployment/testing, not exploitable post-deployment.

Recommendation: Add interface validation in constructor to verify USDSC implements required interfaces:

```
constructor(address usdscAddress, /*...*/) {
    if (usdscAddress == address(0)) revert ZeroAddress();
    try IERC20(usdscAddress).totalSupply() returns (uint256) {}
    catch { revert("Invalid USDSC: missing IERC20"); }
    try IMYieldToOne(usdscAddress).yield() returns (uint256) {}
    catch { revert("Invalid USDSC: missing IMYieldToOne"); }
    USDSC_ADDRESS = usdscAddress;
}
```

STA-11

Boost Rewards System Incompatible with Double-Entry Point Tokens

• Informational ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

This maybe an overkill. We already maintain token list – activeBoostTokens gating the claim loop
Keeper service is in our control where we are confident we won't be sending any double entry tokens.
We'll document that we don't support double entry point tokens

File(s) affected: `src/vaults/earn/BoostRewardsLib.sol`, `src/vaults/earn/EarnVaultUpgradeable.sol`,
`src/vaults/earn/EarnVault.sol`

Description: The boost rewards system tracks each token address separately in `activeBoostTokens` array with corresponding `boostClaimReserve[token]` accounting. However, it is incompatible with double-entry point tokens (rare tokens with two contract addresses pointing to the same underlying storage, like the old UST token). If both addresses of such a token are added as boost rewards, the accounting will become desynchronized from the actual token balance, leading to DoS and confusion.

```
// BoostRewardsLib.sol:44-55
uint256 bal = IERC20(token).balanceOf(address(this));

if (totalPrincipal == 0) {
    if (bal < amount) revert IEarnVaultEventsAndErrors.InsufficientBoostTokenBalance();
    IERC20(token).safeTransfer(treasury, amount);
    return;
}

// Check: balance must cover existing claims + new amount
if (bal < boostClaimReserve[token] + amount) revert
IEarnVaultEventsAndErrors.InsufficientBoostClaimReserve();
```

```
// BoostRewardsLib.sol:65-72
boostClaimReserve[token] += amount;

// Track active boost tokens (only add if not already tracked)
if (boostTokenIndex[token] == 0) {
    activeBoostTokens.push(token);
    boostTokenIndex[token] = activeBoostTokens.length; // 1-based index
}
```

Exploit Scenario:

A double-entry point token exists with addresses `TokenA` and `TokenB` sharing the same underlying storage. Admin adds `TokenA` as boost reward (100 tokens distributed, `boostClaimReserve[TokenA] = 100`). Admin unknowingly adds `TokenB` thinking it's different and distributes 50 tokens (`boostClaimReserve[TokenB] = 50`). Users can claim via both `TokenA` and `TokenB` independently as `activeBoostTokens = [TokenA, TokenB]` loops through both, each with separate `boostClaimReserve` tracking, but both point to the same actual token balance. This creates accounting confusion where `boostClaimReserve[A] + boostClaimReserve[B]` may not match actual balance, potentially causing DoS on boost reward distribution with unexpected reverts when claiming. Requires extremely rare token type and admin error adding both addresses, but can cause DoS and accounting confusion without direct fund loss.

Recommendation: Document the limitation and implement token whitelist with vetting procedures:

```
/// @dev WARNING: Incompatible with double-entry point tokens
mapping(address => bool) public approvedBoostTokens;

function approveBoostToken(address token) external onlyOwner {
    require(!approvedBoostTokens[token], "Already approved");
    approvedBoostTokens[token] = true;
}

function onBoostReward(address token, uint256 amount) external {
    require(approvedBoostTokens[token], "Token not approved");
    // ... rest of function
}
```

STA-12

Lack of Support for Fee-on-Transfer and Rebasing Tokens

• Informational ⓘ Acknowledged

i

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

Fee-On-Transfer and Rebasing tokens are not supported for Boosted rewards.

File(s) affected: EarnVault.sol , EarnVaultUpgradeable.sol , BoostRewardsLib.sol

Description: A fee-on-transfer token is an ERC20 that, when transferred, burns or redirects a portion of the transferred amount to an address different from the ordinary recipient. Meanwhile, a rebasing token is one for which an address's balance can change without any intervention from a user (e.g. to model accumulation of interest).

Considering BoostRewardsLib is used to send arbitrary tokens to EarnVault , it is possible in principle that one of these tokens could be a fee-on-transfer token or a rebasing token. However, in each case, such a token would be incompatible with EarnVault 's accounting system.

- The distributeBoostReward() function checks `bal < boostClaimReserve[token] + amount` at BoostRewardsLib.sol#L55 . However, if a keeper transfers `amount` tokens while the vault receives `amount - fee` tokens, this check will likely fail, causing the transaction to revert.
- Whereas the `boostClaimReserve[token]` value is used to model the amount of the token in the EarnVault that is available to be claimed from the contract, a rebasing token's balance can be expected to diverge from this value, potentially leaving users unable to claim the amount they expect to be entitled to.

Recommendation: If fee-on-transfer and/or rebasing tokens are desired to be compatible with EarnVault , the logic in onBoostReward() should be re-written in order to accommodate these cases. If not, it should be explicitly documented that these cases are not allowed and to be avoided by privileged actors.

STA-13

depositWithPermit() Cannot Be Used to Delegate Transaction Signing

• Informational ⓘ Fixed

✓

Update
Marked as "Fixed" by the client.
Addressed in: a218e7c50ccf477d43c59c835634e7f0115155dc .

File(s) affected: EarnVault.sol , EarnVaultUpgradeable.sol

Description: Currently, the EarnVault contract exposes two options for deposits, either calling the deposit() function directly, or using depositWithPermit() which relies on an additional EIP-712 signature to increase the token allowance without requiring a prior call to approve() from the token holder. Both functions are otherwise identical, and only deal with deposits in USDSC. It is therefore not clear why both functions are needed, given that users are free to choose the unrestricted one, and **in both cases the caller must be the token owner**, and thus must already hold native tokens.

Recommendation: Consider allowing deposits with another user's tokens using depositWithPermit() . The token owner would then not be required to hold native tokens to call approve() , and may allow other parties to execute the deposit and take token ownership.

Auditor Suggestions

S1 Designate BoostRewardsLib Functions internal

Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `b09924b8481ce68c87e67f17f2081414a058147f` .

Description: Currently, the functions in `BoostRewardsLib` are marked `external` . This causes the library to be deployed as a separate contract, which is most appropriate when it is to be shared between multiple contracts to, e.g., save deployment size.

`BoostRewardsLib` is being used in both `EarnVault` and `EarnVaultUpgradeable` . However, the use of `internal` would be more gas-efficient due to inlining of the function bytecode.

Recommendation: If the network on which contracts are deployed has a high enough contract size limit, mark the functions in `BoostRewardsLib` as `internal` , especially if only one vault is to be deployed.

S2 Avoid Acronyms in Variable Naming

Acknowledged



Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

```
does not seem super necessary to rename gi and ui variables.
```

Description: Currently, acronyms are being used in variable names for memory variables. Consider, for example, the use of `gi` and `ui` in `EarnVault.claimable()` . However, this can make the code difficult to read at a glance.

Recommendation: Instead of using acronyms, consider using underscores (e.g. `groupIndex_`) to distinguish memory variables from storage variables.

S3 General Improvements

Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `75ac2db44099efb033653f28201e1a559ee5a4e2` .

Description: The following are a list of general improvements that can save gas and/or increase readability/maintainability.

- `RewardDistributor.distribute()` : Combine the USDSC transfer of `feeToStartale` and `toStartaleExtra` to the treasury to save gas.
- `RewardDistributor._calculateSplit()` : Avoid using magic numbers and create a `constant` contract field `BPS_DENOMINATOR = 10_000` .
- `BoostRewardsLib.claimBoostReward()` : The `return claimedAmount;` expression is not needed.
- `EarnVault.setBlacklisted` : This function can emit `BlacklistStatusChanged` even if the value has not changed. The same applies to `setPauser()` and `setTreasury()` .

Recommendation: Address the suggested improvements.

S4 Code Duplication

Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `82c5f67b2d46d4a8858be416a8181ed3572279b1` .

Description: There are a number of instances where the functionality of code is duplicated. Remediating these instances would result in increased readability/maintainability and reduced deployment causes.

- `RewardDistributor._calculateSplit()` : Call `_supplyBase()` instead of recalculating `S_base` .
- `EarnVault.depositWithPermit()` : This function is nearly identical to `deposit()` . Consider extracting duplicate code to a `_deposit()` helper function.
- `EarnVault.claim()` : The code that settles USDSC claims should be extracted into a `_claimUSDSC()` helper function, and similarly with `_claimBoostRewards()` to avoid code duplication with `withdraw()` .

- 4. `EarnVault.totalValue()`: This function is nearly identical to `claimable()`, and could be simplified by returning `p + this.claimable(user)`. The same applies to parts of `getUserInfo()`.
- 5. `EarnVault._settleBoost()`: This code is already executed in `BoostRewardsLib.claimBoostRewards`. Thus, all calls to this function can be avoided, and this function removed entirely.

Recommendation: Consolidate instances of duplicate code.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/StartaleGroup/stablecoin-contracts`

- `469...571 ./src/distributor/RewardRedistributor.sol`
- `2b3...a37 ./src/interfaces/vaults/earn/IEarnVault.sol`
- `133...42c ./src/interfaces/vaults/earn/IEarnVaultEventsAndErrors.sol`
- `419...8ea ./src/vaults/earn/BoostRewardsLib.sol`
- `7d3...b6d ./src/vaults/earn/EarnVault.sol`
- `957...3a6 ./src/vaults/earn/EarnVaultStorageBase.sol`
- `de0...c40 ./src/vaults/earn/EarnVaultUpgradeable.sol`

Test Suite Results

The test suite has significantly increased in size from the initial audit to the post-fixes version.

```
Ran 4 tests for test/unit/USDSC.mock.t.sol:UnitUSDSC
[PASS] test_claimYield() (gas: 250567)
Logs:
  currentIndex 0
  currentEarnerRate 0
  newCurrentIndex 1056091682480
  usdsc.balanceOf(yieldRecipient) 1000000000000000000

[PASS] test_freezing() (gas: 48094)
[PASS] test_initialize() (gas: 31322)
[PASS] test_yieldRecipientChange() (gas: 38470)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 283.95ms (52.76ms CPU time)
```

```
Ran 10 tests for test/invariants/EarnVaultInvariants.t.sol:EarnVaultInvariants
[PASS] test_BalanceConservationInvariant() (gas: 202954)
[PASS] test_BoostCarryCorrectness() (gas: 1155695)
[PASS] test_BoostInvariant() (gas: 1153647)
[PASS] test_FairnessInvariant() (gas: 375038)
[PASS] test_GlobalIndexInvariant() (gas: 285031)
[PASS] test_MultiTokenBoostRewards() (gas: 3022494)
[PASS] test_PartialWithdrawAfterBoost() (gas: 1202605)
[PASS] test_PrincipalSumInvariant() (gas: 377430)
[PASS] test_ResetOnClaimWithdrawInvariant() (gas: 238630)
[PASS] test_USDSCBalanceInvariant() (gas: 211010)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 288.36ms (62.22ms CPU time)
```

```
Ran 17 tests for test/unit/EarnVaultUpgrades.t.sol:EarnVaultUpgradesTest
[PASS] test_CannotReinitializeV2() (gas: 68053)
[PASS] test_CannotReinitializeV3() (gas: 116714)
[PASS] test_CannotSendETHToVaultV1() (gas: 22197)
[PASS] test_CannotSendETHToVaultV2() (gas: 76424)
[PASS] test_CannotSendETHToVaultV3() (gas: 125103)
[PASS] test_EthRejectionViaReceiveAndFallback() (gas: 37730)
[PASS] test_EthRejectionWorksThroughUpgradeChain() (gas: 340994)
[PASS] test_ImplementationUpdatesCorrectly() (gas: 165880)
[PASS] test_StorageConsistencyAcrossUpgrades() (gas: 468738)
[PASS] test_V1ToV2ToV3ChainUpgrade() (gas: 374411)
[PASS] test_V1ToV2UpgradePreservesComplexState() (gas: 396771)
[PASS] test_V1ToV2UpgradeWithInitialization() (gas: 291099)
[PASS] test_V2ToV3UpgradeWithComplexState() (gas: 603837)
[PASS] test_V2ToV3UpgradeWithInitialization() (gas: 405495)
[PASS] test_V3AutoCompound() (gas: 447339)
[PASS] test_V3FeeCollection() (gas: 435632)
[PASS] test_V3OverriddenDepositWithAutoCompound() (gas: 459141)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 294.85ms (65.39ms CPU time)
```

```
Ran 20 tests for test/unit/EarnVaultUpgradeableUsingOz.t.sol:EarnVaultUpgradeableUsingOzTest
[PASS] test_BasicDepositAndWithdraw() (gas: 262859)
[PASS] test_BlacklistFunctionality() (gas: 356327)
[PASS] test_BlacklistPreservedAcrossUpgrades() (gas: 443801)
[PASS] test_CannotSendETHToVault() (gas: 22285)
[PASS] test_Initialize() (gas: 59191)
[PASS] test_InitializeZeroOwner() (gas: 5354131)
[PASS] test_InitializeZeroPauser() (gas: 5354174)
[PASS] test_InitializeZeroTreasury() (gas: 5354207)
[PASS] test_InitializeZeroUSDSC() (gas: 5354043)
[PASS] test_InitializeZeroYieldRedistributor() (gas: 5354185)
[PASS] test_PauseFunctionality() (gas: 249465)
[PASS] test_PausePreservedAcrossUpgrades() (gas: 250697)
[PASS] test_SweepNativeOnlyOwner() (gas: 159146)
[PASS] test_SweepNativeWorks() (gas: 148853)
[PASS] test_V1ToV2ToV3ChainUpgrade() (gas: 376219)
[PASS] test_V1ToV2Upgrade() (gas: 288830)
[PASS] test_V2ToV3Upgrade() (gas: 406452)
[PASS] test_V3AutoCompound() (gas: 446542)
[PASS] test_V3FeeCollection() (gas: 434811)
[PASS] test_YieldDistribution() (gas: 298036)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in 294.36ms (66.03ms CPU time)
```

```
Ran 23 tests for test/unit/EarnVaultBoostTest.t.sol:EarnVaultBoostTest
[PASS] test_BoostRewardDistribution() (gas: 422729)
[PASS] test_CannotClaimBoostRewardsMultipleTimes() (gas: 454251)
[PASS] test_ClaimAutomaticallyClaimsAllRewards() (gas: 428696)
[PASS] test_ClaimBoostReward_GlobalIndexEqualsUserIndex() (gas: 360825)
[PASS] test_ClaimBoostReward_GlobalIndexGreaterThanUserIndex() (gas: 402915)
[PASS] test_ClaimBoostReward_MultipleUsersWithDifferentPrincipals() (gas: 540910)
[PASS] test_ClaimBoostReward_WithZeroPrincipal() (gas: 140229)
[PASS] test_DepositSettlesBoostRewards_NegativeTest_WithoutFixWouldFail() (gas: 432199)
[PASS] test_DepositSettlesBoostRewards_PreventsRetroactiveAccrual() (gas: 461690)
[PASS] test_GetClaimableBoostReward_WithZeroPrincipal() (gas: 356841)
[PASS] test_MultipleTokenBoostRewards() (gas: 573305)
[PASS] test_NoDepositsBoostRewardToTreasury() (gas: 95289)
[PASS] test_NoDoubleAccrualInClaim() (gas: 380830)
[PASS] test_NoDoubleAccrualInWithdraw() (gas: 356980)
```



```
[PASS] test_ProportionalBoostDistribution() (gas: 423608)
[PASS] test_RemoveBoostRewardToken_CanReAddAfterRemoval() (gas: 464984)
[PASS] test_RemoveBoostRewardToken_CannotRemoveWithPendingClaims() (gas: 360561)
[PASS] test_RemoveBoostRewardToken_FromMiddleOfArray() (gas: 1449664)
[PASS] test_RemoveBoostRewardToken_NonExistentToken() (gas: 17310)
[PASS] test_RemoveBoostRewardToken_OnlyOwner() (gas: 376268)
[PASS] test_RemoveBoostRewardToken_ReducesLoopIterations() (gas: 643335)
[PASS] test_RemoveBoostRewardToken_Success() (gas: 567296)
[PASS] test_UserClaimBoostRewards() (gas: 361073)
Suite result: ok. 23 passed; 0 failed; 0 skipped; finished in 316.40ms (69.59ms CPU time)
```

Ran 41 tests for test/unit/EarnVaultUpgradeableSimple.t.sol:EarnVaultUpgradeableSimpleTest

```
[PASS] test_AdminCannotSweepNative() (gas: 159104)
[PASS] test_AdminFunctionsWorkAfterUpgrade() (gas: 5593450)
[PASS] test_BlacklistStatusPreservedAcrossUpgrades() (gas: 12279615)
[PASS] test_CannotInitializeImplementationDirectly() (gas: 5311703)
[PASS] test_CannotInitializeTwice() (gas: 31255)
[PASS] test_CannotSendETHToVaultV1() (gas: 22219)
[PASS] test_CannotSendETHToVaultV2() (gas: 5570064)
[PASS] test_CannotSendETHViaFallback() (gas: 22489)
[PASS] test_CannotSendETHViaReceive() (gas: 22286)
[PASS] test_ClaimWorksBeforeAndAfterUpgrade() (gas: 5767419)
[PASS] test_ComplexStatePreservationAcrossUpgrades() (gas: 13078611)
[PASS] test_DepositWorksBeforeAndAfterUpgrade() (gas: 5787383)
[PASS] test_EthRejectionScenarios() (gas: 174296)
[PASS] test_EthRejectionWorksAfterUpgrade() (gas: 5723709)
[PASS] test_InitializationSetsCorrectValues() (gas: 59190)
[PASS] test_OnBoostRewardStabilityAcrossUpgrades() (gas: 12913590)
[PASS] test_OnYieldStabilityAcrossUpgrades() (gas: 12082211)
[PASS] test_OnlyAdminCanUpgrade() (gas: 5543837)
[PASS] test_PauseWorksAfterUpgrade() (gas: 5801401)
[PASS] test_RecoverERC20_CannotReenter() (gas: 236747)
[PASS] test_RecoverERC20_ReentrancyProtection() (gas: 240745)
[PASS] test_SelfDestructorCodeRemainsAfterDestruct() (gas: 101784)
[PASS] test_SweepNativeAfterUpgrade() (gas: 5698487)
[PASS] test_SweepNativeInsufficientBalance() (gas: 56990)
[PASS] test_SweepNativeOnlyOwner() (gas: 159146)
[PASS] test_SweepNativeWorks() (gas: 148809)
[PASS] test_SweepNativeZeroAddress() (gas: 108318)
[PASS] test_UpgradeDoesNotResetRolesOrStates() (gas: 5783281)
[PASS] test_UpgradeEmitsEvent() (gas: 5529388)
[PASS] test_UpgradePreservesBalances() (gas: 5844579)
[PASS] test_UpgradePreservesBasicState() (gas: 5781413)
[PASS] test_UpgradeWhilePaused() (gas: 5755303)
[PASS] test_UpgradeWithAccumulatedYield() (gas: 5872633)
[PASS] test_UserOperationsStabilityAcrossUpgrades() (gas: 13207082)
[PASS] test_V2NewFeaturesWork() (gas: 5601408)
[PASS] test_V2NewFunctionality() (gas: 5601553)
[PASS] test_ViewFunctionsWorkAfterUpgrade() (gas: 5907694)
[PASS] test_WithdrawWorksBeforeAndAfterUpgrade() (gas: 5775230)
[PASS] test_YieldDistributionWorksBeforeAndAfterUpgrade() (gas: 5863443)
[PASS] test_getImplementation() (gas: 16484)
[PASS] test_initialize() (gas: 45830)
```

Suite result: ok. 41 passed; 0 failed; 0 skipped; finished in 315.83ms (88.25ms CPU time)

Ran 44 tests for test/unit/SUSDSCVaultUpgrade.t.sol:SUSDSCVaultUpgradeTest

```
[PASS] test_AccessControlWorksAfterUpgrade() (gas: 3117563)
[PASS] test_CannotInitializeImplementationDirectly() (gas: 2987487)
[PASS] test_CannotInitializeTwice() (gas: 23929)
[PASS] test_CannotUpgradeToZeroAddress() (gas: 22354)
[PASS] test_Decimals_ConsistentAfterUpgrade() (gas: 3077767)
[PASS] test_Decimals_MatchesAsset() (gas: 32091)
[PASS] test_DepositWorksAfterUpgrade() (gas: 3182615)
[PASS] test_InitializationSetsCorrectValues() (gas: 49475)
[PASS] test_Mint_MaxMint() (gas: 137325)
[PASS] test_Mint_MultipleUsers() (gas: 185155)
[PASS] test_Mint_Success() (gas: 140169)
[PASS] test_Mint_ToReceiver() (gas: 140416)
[PASS] test_Mint_WithYield() (gas: 410228)
[PASS] test_MultipleSequentialUpgrades() (gas: 9240100)
[PASS] test_OnlyAdminCanUpgrade() (gas: 3074538)
[PASS] test_PauseWorksAfterUpgrade() (gas: 3088783)
```



```
[PASS] test_ProxyAdminOwnerCannotBeChanged() (gas: 17147)
[PASS] test_RecoverNonAssetERC20_Success() (gas: 1059247)
[PASS] test_Revert_Mint_InsufficientAssets() (gas: 42661)
[PASS] test_Revert_Mint_WhenPaused() (gas: 48234)
[PASS] test_Revert_RecoverNonAssetERC20_AssetToken() (gas: 26557)
[PASS] test_Revert_RecoverNonAssetERC20_NotAdmin() (gas: 1047034)
[PASS] test_Revert_RecoverNonAssetERC20_ZeroAmount() (gas: 863315)
[PASS] test_Revert_RecoverNonAssetERC20_ZeroRecipient() (gas: 861158)
[PASS] test_Revert_RecoverNonAssetERC20_ZeroTokenAddress() (gas: 24452)
[PASS] test_Revert_Withdraw_InsufficientShares() (gas: 132285)
[PASS] test_Revert_Withdraw_WhenPaused() (gas: 155313)
[PASS] test_UpgradeDoesNotResetRoles() (gas: 3139143)
[PASS] test_UpgradeEmitsEvent() (gas: 3068303)
[PASS] test_UpgradePreservesBalances() (gas: 3195803)
[PASS] test_UpgradePreservesBasicState() (gas: 3096674)
[PASS] test_UpgradePreservesPausedState() (gas: 3108351)
[PASS] test_UpgradePreservesPricePerShare() (gas: 3401631)
[PASS] test_UpgradePreservesRoles() (gas: 3098396)
[PASS] test_UpgradeWhilePaused() (gas: 3171667)
[PASS] test_UpgradeWithAccumulatedYield() (gas: 3398405)
[PASS] test_UpgradeWithMultipleDepositors() (gas: 3217885)
[PASS] test_V2NewFunctionality() (gas: 3229703)
[PASS] test_ViewFunctionsWorkAfterUpgrade() (gas: 3225655)
[PASS] test_WithdrawWorksAfterUpgrade() (gas: 3144197)
[PASS] test_Withdraw_Success() (gas: 161896)
[PASS] test_Withdraw_ToReceiver() (gas: 130019)
[PASS] test_Withdraw_WithApproval() (gas: 152586)
[PASS] test_Withdraw_WithYield() (gas: 384838)
Suite result: ok. 44 passed; 0 failed; 0 skipped; finished in 318.73ms (85.16ms CPU time)
```

Ran 53 tests for test/unit/SUSDSCVault.t.sol:UnitSUSDSCVault

```
[PASS] test_access_control_roles() (gas: 36844)
[PASS] test_consecutive_yield_distributions() (gas: 165675)
[PASS] test_constructor() (gas: 38658)
[PASS] test_decimals() (gas: 22034)
[PASS] test_deposit_after_yield_reduces_shares() (gas: 203953)
[PASS] test_deposit_multiple_users() (gas: 254366)
[PASS] test_deposit_single_user() (gas: 144166)
[PASS] test_high_pps_yield_arrival_reduces_mint_shares() (gas: 210025)
[PASS] test_maxDeposit_maxMint() (gas: 13886)
[PASS] test_maxWithdraw_maxRedeem() (gas: 141427)
[PASS] test_mint_functionality() (gas: 131918)
[PASS] test_multiple_users_yield_distribution_scenario() (gas: 309957)
[PASS] test_pause_functionality() (gas: 142311)
[PASS] test_previewFunctions_consistency() (gas: 64324)
[PASS] test_recoverNonAssetERC20_admin_role_changes() (gas: 861332)
[PASS] test_recoverNonAssetERC20_does_not_affect_vault_operations() (gas: 977087)
[PASS] test_recoverNonAssetERC20_edge_cases() (gas: 808874)
[PASS] test_recoverNonAssetERC20_multiple_tokens() (gas: 1576733)
[PASS] test_recoverNonAssetERC20_partial_recovery() (gas: 814474)
[PASS] test_recoverNonAssetERC20_success() (gas: 798391)
[PASS] test_redeem_after_multiple_yield_distributions() (gas: 160104)
[PASS] test_redeem_after_yield_distribution() (gas: 144065)
[PASS] test_redeem_all_shares() (gas: 144397)
[PASS] test_redeem_consistency_with_convert_functions() (gas: 138000)
[PASS] test_redeem_multiple_times_same_user() (gas: 170793)
[PASS] test_redeem_partial_shares() (gas: 164413)
[PASS] test_redeem_precision() (gas: 133736)
[PASS] test_redeem_to_different_receiver() (gas: 146880)
[PASS] test_redeem_with_approval() (gas: 173328)
[PASS] test_redeem_with_yield_between_deposits() (gas: 224508)
[PASS] test_revert_deposit_insufficient_allowance() (gas: 68195)
[PASS] test_revert_deposit_insufficient_balance() (gas: 73519)
[PASS] test_revert_deposit_when_paused() (gas: 84218)
[PASS] test_revert_mint_when_paused() (gas: 84218)
[PASS] test_revert_pause_unauthorized() (gas: 14206)
[PASS] test_revert_recoverNonAssetERC20_asset_token() (gas: 17485)
[PASS] test_revert_recoverNonAssetERC20_insufficient_balance() (gas: 783895)
[PASS] test_revert_recoverNonAssetERC20_invalid_token_address() (gas: 19817)
[PASS] test_revert_recoverNonAssetERC20_not_admin() (gas: 790331)
[PASS] test_revert_recoverNonAssetERC20_zero_amount() (gas: 783232)
[PASS] test_revert_recoverNonAssetERC20_zero_to_address() (gas: 782855)
```

```
[PASS] test_revert_recoverNonAssetERC20_zero_token_address() (gas: 15402)
[PASS] test_revert_redeem_insufficient_shares() (gas: 129366)
[PASS] test_revert_redeem_when_paused() (gas: 156170)
[PASS] test_revert_withdraw_insufficient_shares() (gas: 133297)
[PASS] test_revert_withdraw_when_paused() (gas: 156063)
[PASS] test_very_small_deposits() (gas: 703095)
[PASS] test_withdraw_functionality() (gas: 157478)
[PASS] test_yield_distribution_increases_pps() (gas: 214823)
[PASS] test_yield_distribution_proportional_gains() (gas: 214844)
[PASS] test_zero_deposit_allowed() (gas: 61040)
[PASS] test_zero_mint_allowed() (gas: 53199)
[PASS] test_zero_total_supply_yield_distribution() (gas: 206348)
Suite result: ok. 53 passed; 0 failed; 0 skipped; finished in 350.23ms (185.84ms CPU time)
```

Ran 14 tests for test/fork/USDSC.sepolia.fork.t.sol:ForkUSDSC

```
[PASS] test_adminCanSetYieldRecipient() (gas: 74942)
```

Logs:

```
Setting M token balance – desired: 1000000000
Scaled input: 256000000000
Old Yield Recipient: 0x2e4B5e2b7862195db1AcDd95E8328c38BC607260
New Yield Recipient: 0x6bb99143628a9278d68c0F95a9e3506D3a5E821a
```

```
[PASS] test_claimYield() (gas: 78585)
```

Logs:

```
Setting M token balance – desired: 1000000000
Scaled input: 256000000000
Initial Yield: 353284457
Yield Recipient: 0x2e4B5e2b7862195db1AcDd95E8328c38BC607260
Claimed Yield: 353284457
Recipient Balance Increase: 353284457
```

```
[PASS] test_contractInfo() (gas: 49613)
```

Logs:

```
Setting M token balance – desired: 1000000000
Scaled input: 256000000000
USDSC Name: USDRtestnet
USDSC Symbol: USDR
USDSC Decimals: 6
USDSC Total Supply: 1004831541346
USDSC Yield Recipient: 0x2e4B5e2b7862195db1AcDd95E8328c38BC607260
```

```
[PASS] test_contractInfoWithMToken() (gas: 48364)
```

Logs:

```
Setting M token balance – desired: 1000000000
Scaled input: 256000000000
=== Enhanced Contract State ===
USDSC Address: 0x7E426d026f604d1c47b50059752122d8ab1E2C28
M Token Address: 0x866A2BF4E572CbcF37D5071A7a58503Bfb36be1b
SwapFacility Address: 0x34F141dACB2DeF72D2196a473C585830af8B4004
Admin Address: 0x77001610a4fD68548B80E49226c02a99c3b6Ae14
```

```
=== Token Balances ===
```

```
Admin M Balance: 1000000000
Admin USDSC Balance: 500000000
SwapFacility M Balance: 0
USDSC Contract M Balance: 1005184825803
USDSC Total Supply: 1004831541346
```

```
[PASS] test_contractState() (gas: 63514)
```

Logs:

```
Setting M token balance – desired: 1000000000
Scaled input: 256000000000
=== USDSC Contract State ===
Address: 0x7E426d026f604d1c47b50059752122d8ab1E2C28
Name: USDRtestnet
Symbol: USDR
Decimals: 6
Total Supply: 1004831541346
Yield Recipient: 0x2e4B5e2b7862195db1AcDd95E8328c38BC607260
Available Yield: 353284457
Admin Balance: 500000000
Contract Balance: 0
```

[PASS] test_nonAdminCannotSetYieldRecipient() (gas: 20743)

Logs:

Setting M token balance – desired: 1000000000

Scaled input: 256000000000

[PASS] test_readMTokenCurrentIndex() (gas: 11783)

Logs:

Setting M token balance – desired: 1000000000

Scaled input: 256000000000

Current Index: 1034484678150

[PASS] test_roleBasedAccess() (gas: 71176)

Logs:

Setting M token balance – desired: 1000000000

Scaled input: 256000000000

Yield Recipient Manager Role:

0x4a5e9eb1ba56d04185ff75ebf0f4f42a3d7c88c35b4a90fa278437e0c9bdceca

[PASS] test_tokenTransfers() (gas: 227793)

Logs:

Setting M token balance – desired: 1000000000

Scaled input: 256000000000

=== Initial Balances ===

Zelda Balance: 1000000000

Admin Balance: 500000000

Bob Balance: 0

=== After Zelda -> Bob Transfer ===

Transferred Amount: 100000000

Bob's New Balance: 100000000

Zelda's New Balance: 900000000

=== After Admin -> Charlie Transfer ===

Transferred Amount: 50000000

Charlie's New Balance: 50000000

Admin's New Balance: 450000000

=== After Approve/TransferFrom Test ===

Approved Amount: 75000000

TransferFrom Amount: 30000000

User's New Balance: 30000000

Remaining Allowance: 45000000

=== Total Supply Check ===

Total Supply (should still be earlierTotalSupply): 1004831541346

[PASS] test_unwrapFunctionality() (gas: 343626)

Logs:

Setting M token balance – desired: 1000000000

Scaled input: 256000000000

=== Unwrap Test ===

[PASS] Non-SwapFacility correctly cannot call unwrap

[PASS] SwapFacility can call unwrap (reverted as expected due to no tokens)

SwapFacility USDSC Balance after deal: 50000000

[PASS] SwapFacility unwrap succeeded – USDSC burned and M tokens received

[PASS] test_wrapFunctionality() (gas: 155712)

Logs:

Setting M token balance – desired: 1000000000

Scaled input: 256000000000

=== Wrap Test ===

Admin M Balance: 1000000000

[PASS] Non-SwapFacility correctly cannot call wrap

Admin M Balance after vm.store: 1000000000

SwapFacility M Balance after transfer: 100000000

[PASS] SwapFacility successfully wrapped M tokens to USDSC!

Bob's USDSC balance after wrap: 100000000

USDSC total supply after wrap: 1004931541346

M Token balance of USDSC: 1005284825802

[PASS] SwapFacility address correctly configured: 0x34F141dACB2DeF72D2196a473C585830af8B4004

[PASS] M Token address correctly configured: 0x866A2BF4E572CbcF37D5071A7a58503Bfb36be1b

[PASS] test_wrapUnwrapAccessControl() (gas: 119694)

Logs:

Setting M token balance – desired: 1000000000

```
Scaled input: 256000000000
=== Access Control Test ===
[PASS] All unauthorized callers correctly rejected for wrap()
[PASS] All unauthorized callers correctly rejected for unwrap()
[PASS] SwapFacility can call wrap() (reverted for balance reasons, not access)
[PASS] SwapFacility can call unwrap() (reverted for balance reasons, not access)

[PASS] test_yieldAmount() (gas: 24880)
Logs:
  Setting M token balance – desired: 1000000000
  Scaled input: 256000000000
  Current Yield Available: 353284457

[PASS] test_yieldRecipient() (gas: 19440)
Logs:
  Setting M token balance – desired: 1000000000
  Scaled input: 256000000000
  Current Yield Recipient: 0x2e4B5e2b7862195db1AcDd95E8328c38BC607260

Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 4.66s (2.86s CPU time)
```

Ran 84 tests for test/unit/EarnVault.t.sol:EarnVaultTest

```
[PASS] test_Asset() (gas: 11374)
[PASS] test_AssetViewFunction() (gas: 11775)
[PASS] test_BasicClaim() (gas: 210034)
[PASS] test_BasicDeposit() (gas: 174608)
[PASS] test_BlacklistBlocksDepositWithPermit() (gas: 48157)
[PASS] test_BlacklistFunctionality() (gas: 254457)
[PASS] test_BlacklistedUserCannotAccessBoostRewards() (gas: 1082986)
[PASS] test_BoostRewardAccessControl() (gas: 1071061)
[PASS] test_BoostRewardKeeperCanBeChanged() (gas: 1068322)
[PASS] test_BoostRewardSpecificErrors() (gas: 971724)
[PASS] test_ClaimNoRewards() (gas: 47334)
[PASS] test_ClaimWithNoYield() (gas: 154543)
[PASS] test_ClaimableNoDeposits() (gas: 16041)
[PASS] test_ClaimableViewFunction() (gas: 223426)
[PASS] test_CompleteUserJourney() (gas: 398116)
[PASS] test_ComprehensiveYieldCalculationOverPeriods() (gas: 485000)
[PASS] test_DepositBackAfterWithdrawalIsFair() (gas: 282872)
[PASS] test_DepositWithPermitFailure() (gas: 22655)
[PASS] test_DepositWithPermit_TriggersSettleWithExistingPrincipal() (gas: 232778)
[PASS] test_DepositZeroAmount() (gas: 16598)
[PASS] test_DistributorManagement() (gas: 253134)
[PASS] test_DynamicPrincipalChanges() (gas: 244209)
[PASS] test_EmergencySweepToTreasury() (gas: 231503)
[PASS] test_FullWithdrawNoYield() (gas: 135355)
[PASS] test_FundingInvariantNoDeposits() (gas: 102373)
[PASS] test_FundingInvariantWithMultipleUsers() (gas: 267921)
[PASS] test_FundsSentWithoutOnYield() (gas: 202057)
[PASS] test_GetAllClaimables() (gas: 2070396)
[PASS] test_GetAllClaimablesNoBoostRewards() (gas: 189516)
[PASS] test_GetClaimableBoostRewardNoRewards() (gas: 25443)
[PASS] test_GetUserInfoNoDeposits() (gas: 23247)
[PASS] test_GetVaultStats() (gas: 157304)
[PASS] test_IndexSystemPreventsDoubleClaiming() (gas: 267869)
[PASS] test_IndexSystemTracksUserPositions() (gas: 251926)
[PASS] test_MultiUserDeposit() (gas: 230887)
[PASS] test_MultiUserDifferentTimings() (gas: 273994)
[PASS] test_MultipleClaims() (gas: 228132)
[PASS] test_MultipleDifferentBoostTokens_ByKeeper() (gas: 3112152)
[PASS] test_MultipleSmallWithdrawalsNoDoubleClaiming() (gas: 269437)
[PASS] test_OnBoostRewardZeroAmount() (gas: 169960)
[PASS] test_OnYieldWithExcessiveAmount() (gas: 179450)
[PASS] test_OnYieldWithIncorrectAmount() (gas: 203049)
[PASS] test_OnYieldZeroAmount() (gas: 159977)
[PASS] test_OnlyOwnerCanManageBlacklist() (gas: 16490)
[PASS] test_OwnershipTransferAcceptance() (gas: 50439)
[PASS] test_OwnershipTransferCancellation() (gas: 37344)
[PASS] test_OwnershipTransferEvents() (gas: 36624)
[PASS] test_OwnershipTransferInitiation() (gas: 48397)
[PASS] test_PartialWithdrawNoYield() (gas: 179058)
[PASS] test_PartialWithdrawWithYield() (gas: 210189)
```



```
[PASS] test_PauseAccessControl() (gas: 60716)
[PASS] test_PauseFunctionality() (gas: 164723)
[PASS] test_PauserManagement() (gas: 50138)
[PASS] test_ProportionalYieldDistribution() (gas: 263398)
[PASS] test_RealisticLargeYieldAmount() (gas: 178381)
[PASS] test_RecoverERC20_CannotReenter() (gas: 227630)
[PASS] test_RecoverERC20_ReentrancyProtection() (gas: 231672)
[PASS] test_RenounceOwnership() (gas: 31868)
[PASS] test_RoleAccessControl() (gas: 50011)
[PASS] test_SetBlacklisted() (gas: 36884)
[PASS] test_SetPauser() (gas: 24908)
[PASS] test_SetTreasury() (gas: 22788)
[PASS] test_SetYieldRedistributor() (gas: 24808)
[PASS] test_SingleUserYieldDistribution() (gas: 191825)
[PASS] test_SixHourYieldScenario() (gas: 978054)
[PASS] test_SurplusAfterIncorrectOnYield() (gas: 233960)
[PASS] test_SweepNativeWorks() (gas: 143766)
[PASS] test_SweepSurplusToTreasury() (gas: 237973)
[PASS] test_TotalPrincipalTracking() (gas: 237590)
[PASS] test_TotalValueNoDeposits() (gas: 17779)
[PASS] test_TotalValue_AccuracyWithMultipleYields() (gas: 218783)
[PASS] test_TotalValue_AfterPartialWithdrawal() (gas: 226302)
[PASS] test_TotalValue_WithPrincipalAndYield() (gas: 192396)
[PASS] test_TotalValue_WithPrincipalNoNewYield() (gas: 202821)
[PASS] test_TreasuryManagement() (gas: 32054)
[PASS] test_UnauthorizedOwnershipTransfer() (gas: 55277)
[PASS] test_UnauthorizedYieldCall() (gas: 13780)
[PASS] test_VeryHighYieldPercentage() (gas: 187980)
[PASS] test-WithdrawAll() (gas: 181433)
[PASS] test-WithdrawExceedsPrincipal() (gas: 152508)
[PASS] test-WithdrawWithYield() (gas: 181431)
[PASS] test-WithdrawZeroAmount() (gas: 16576)
[PASS] test_YieldWhenNoDeposits() (gas: 239640)
[PASS] test_ZeroAmountOperations() (gas: 165930)
Suite result: ok. 84 passed; 0 failed; 0 skipped; finished in 13.57s (216.77ms CPU time)
```

Ran 40 tests for

test/integration/RewardRedistributor.integration.t.sol:RewardRedistributorIntegrationTest

```
[PASS] testIntegration_BoostRewardsDistribution() (gas: 2259768)
[PASS] testIntegration_BoostRewardsWithWithdrawals() (gas: 1261042)
[PASS] testIntegration_CarryMechanismAccuracy() (gas: 5047799)
[PASS] testIntegration_ConservationWithRealVaults() (gas: 262344)
[PASS] testIntegration_DepositsWithdrawalsAroundDistribution() (gas: 602514)
[PASS] testIntegration_EarnVaultBlacklistFunctionality() (gas: 200974)
[PASS] testIntegration_EarnVaultBoostRewardAccessControl() (gas: 934657)
[PASS] testIntegration_EarnVaultBoostRewardsEdgeCases() (gas: 948727)
[PASS] testIntegration_EarnVaultBoostTokenRecovery() (gas: 956590)
[PASS] testIntegration_EarnVaultETHRejection() (gas: 24137)
[PASS] testIntegration_EarnVaultEmergencyRecovery() (gas: 906465)
[PASS] testIntegration_EarnVaultFundingInvariant() (gas: 249480)
[PASS] testIntegration_EarnVaultInsufficientPrincipalEdgeCase() (gas: 26276)
[PASS] testIntegration_EarnVaultPauseUnpauseFunctionality() (gas: 108512)
[PASS] testIntegration_EarnVaultRoleManagement() (gas: 121092)
[PASS] testIntegration_EarnVaultSurplusSweep() (gas: 84181)
[PASS] testIntegration_EarnVaultViewFunctionEdgeCases() (gas: 51356)
[PASS] testIntegration_EarnVaultWithdrawalsAfterDistribution() (gas: 510266)
[PASS] testIntegration_EarnVaultZeroAmountEdgeCases() (gas: 24435)
[PASS] testIntegration_EmptyVaultScenarios() (gas: 10561235)
[PASS] testIntegration_EmptyVaultYieldHandling() (gas: 583105)
[PASS] testIntegration_FundingInvariantUnderStress() (gas: 3285183)
[PASS] testIntegration_LargeScaleDistribution() (gas: 388748)
[PASS] testIntegration_MixedWithdrawalsAfterMultipleDistributions() (gas: 609468)
[PASS] testIntegration_MultipleDistributionsWithVaryingTVL() (gas: 827576)
[PASS] testIntegration_ProportionalAllocationWithRealTVLs() (gas: 268711)
[PASS] testIntegration_ProportionalityInvariant() (gas: 1096478)
[PASS] testIntegration_RewardRedistributorAccessControl() (gas: 44505)
[PASS] testIntegration_RewardRedistributorEventEmission() (gas: 264806)
[PASS] testIntegration_RewardRedistributorMaxFeeScenario() (gas: 256867)
[PASS] testIntegration_RewardRedistributorParameterValidation() (gas: 59140)
[PASS] testIntegration_RewardRedistributorPausedDistribution() (gas: 250426)
[PASS] testIntegration_RewardRedistributorPreviewFunctions() (gas: 83284)
```

```
[PASS] testIntegration_RewardRedistributorWithFees() (gas: 271635)
[PASS] testIntegration_RewardRedistributorZeroSupply() (gas: 35398)
[PASS] testIntegration_RewardRedistributorZeroYield() (gas: 140092)
[PASS] testIntegration_SUSDSCVaultPPSMonotonic() (gas: 250584)
[PASS] testIntegration_SUSDSCVaultRedemptionsAfterDistribution() (gas: 368066)
[PASS] testIntegration-WithdrawalOrderingAndInvariants() (gas: 507242)
[PASS] testIntegration_YieldDistributionPrecision() (gas: 1217784)
Suite result: ok. 40 passed; 0 failed; 0 skipped; finished in 80.05s (116.92ms CPU time)
```

Ran 48 tests for test/unit/EarnVaultUpgradeableEdgeCases.t.sol:EarnVaultUpgradeableEdgeCasesTest

```
[PASS] test_DepositWithPermit_RelayerFunctionality() (gas: 7350716)
[PASS] test_DepositWithPermit_TriggersSettleWithExistingPrincipal() (gas: 281993)
[PASS] test_DepositWithPermit_ValidSignature() (gas: 7350780)
[PASS] test_GetAllClaimables_NoPrincipal() (gas: 31239)
[PASS] test_GetAllClaimables_WithBoostRewards() (gas: 2167012)
[PASS] test_GetAllClaimables_WithUSDSCOnly() (gas: 223574)
[PASS] test_OnYieldWithZeroTotalPrincipal() (gas: 94408)
[PASS] test_OwnershipCanBeTransferredViaTwoStepProcess() (gas: 54174)
[PASS] test_RecoverERC20_BoostTokenWithReserve() (gas: 1213089)
[PASS] test_RecoverERC20_NonUSDSCToken() (gas: 892685)
[PASS] test_RecoverERC20_USDSCWhenPaused() (gas: 124483)
[PASS] test_RenounceOwnership() (gas: 43312)
[PASS] test_Revert_DepositWithPermit_Blacklisted() (gas: 106615)
[PASS] test_Revert_DepositWithPermit_ExpiredDeadline() (gas: 7210836)
[PASS] test_Revert_DepositWithPermit_InsufficientBalance() (gas: 7291052)
[PASS] test_Revert_DepositWithPermit_InvalidSignature() (gas: 7229012)
[PASS] test_Revert_DepositWithPermit_Paused() (gas: 101376)
[PASS] test_Revert_DepositWithPermit_PermitFailed() (gas: 80120)
[PASS] test_Revert_DepositWithPermit_ReplayAttack() (gas: 7376530)
[PASS] test_Revert_DepositWithPermit_WrongAmountInSignature() (gas: 7236656)
[PASS] test_Revert_DepositWithPermit_WrongSpenderInSignature() (gas: 7236613)
[PASS] test_Revert_DepositWithPermit_WrongTokenOwner() (gas: 7238521)
[PASS] test_Revert_DepositWithPermit_ZeroAddressTokenOwner() (gas: 7145780)
[PASS] test_Revert_DepositWithPermit_ZeroAmount() (gas: 23869)
[PASS] test_Revert_GetAllClaimables_Blacklisted() (gas: 51773)
[PASS] test_Revert_InitializeWithZeroBoostRewardKeeper() (gas: 5350586)
[PASS] test_Revert_InitializeWithZeroOwner() (gas: 5350443)
[PASS] test_Revert_InitializeWithZeroPauser() (gas: 5350487)
[PASS] test_Revert_InitializeWithZeroRedistributor() (gas: 5350452)
[PASS] test_Revert_InitializeWithZeroTreasury() (gas: 5350452)
[PASS] test_Revert_InitializeWithZeroUSDSC() (gas: 5350355)
[PASS] test_Revert_OnBoostRewardNotRedistributor() (gas: 21590)
[PASS] test_Revert_OnYieldNotRedistributor() (gas: 18978)
[PASS] test_Revert_PauseNotAuthorized() (gas: 18465)
[PASS] test_Revert_RecoverERC20_BoostTokenExceedsAvailable() (gas: 1175767)
[PASS] test_Revert_RecoverERC20_USDSCExceedsSurplus() (gas: 111849)
[PASS] test_Revert_RecoverERC20_USDSCNotPaused() (gas: 29046)
[PASS] test_Revert_RecoverERC20_ZeroAddress() (gas: 20374)
[PASS] test_Revert_SetBoostRewardKeeperZeroAddress() (gas: 19001)
[PASS] test_Revert_SetPauserZeroAddress() (gas: 18981)
[PASS] test_Revert_SetTreasuryZeroAddress() (gas: 18977)
[PASS] test_Revert_SetYieldRedistributorZeroAddress() (gas: 19112)
[PASS] test_Revert_UnpauseNotAuthorized() (gas: 46335)
[PASS] test_SweepSurplusToTreasury() (gas: 261022)
[PASS] test_SweepSurplusToTreasury_NoSurplus() (gas: 207955)
[PASS] test_TotalValue_AccuracyWithMultipleYields() (gas: 254400)
[PASS] test_TotalValue_WithPrincipalAndYield() (gas: 228636)
[PASS] test_TotalValue_WithPrincipalNoNewYield() (gas: 260442)
Suite result: ok. 48 passed; 0 failed; 0 skipped; finished in 80.05s (131.65ms CPU time)
```

Ran 27 tests for test/unit/ERC4626ComplianceTest.t.sol:ERC4626ComplianceTest

```
[PASS] testFuzzDecimalsOverflow(uint8) (runs: 5000, μ: 1718124, ~: 1718474)
[PASS] test_RT_deposit_redeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 535045, ~: 536536)
[PASS] test_RT_deposit_withdraw((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 538068, ~: 539200)
[PASS] test_RT_mint_redeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 535029, ~: 536553)
[PASS] test_RT_mint_withdraw((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 538019, ~: 539181)
[PASS] test_RT_redeem_deposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 535343, ~: 536552)
```

```
[PASS] test_RT_redeem_mint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 535330, ~: 536514)
[PASS] test_RT_withdraw_deposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 538167, ~: 539212)
[PASS] test_RT_withdraw_mint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 538116, ~: 539143)
[PASS] test_asset((address[4],uint256[4],uint256[4],int256)) (runs: 5000, μ: 475394, ~: 477800)
[PASS] test_convertToAssets((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 484339, ~: 486393)
[PASS] test_convertToShares((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 484385, ~: 486480)
[PASS] test_deposit((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 5000, μ: 533004, ~: 534844)
[PASS] test_maxDeposit((address[4],uint256[4],uint256[4],int256)) (runs: 5000, μ: 475791, ~: 478197)
[PASS] test_maxMint((address[4],uint256[4],uint256[4],int256)) (runs: 5000, μ: 475772, ~: 478178)
[PASS] test_maxRedeem((address[4],uint256[4],uint256[4],int256)) (runs: 5000, μ: 476007, ~: 478413)
[PASS] test_maxWithdraw((address[4],uint256[4],uint256[4],int256)) (runs: 5000, μ: 478726, ~: 481010)
[PASS] test_mint((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 5000, μ: 532984, ~: 534808)
[PASS] test_previewDeposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 525098, ~: 526608)
[PASS] test_previewMint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 525107, ~: 526641)
[PASS] test_previewRedeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 527476, ~: 528800)
[PASS] test_previewWithdraw((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 530240, ~: 531463)
[PASS] test_redeem((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 5000, μ: 536127, ~: 537608)
[PASS] test_redeem_zero_allowance((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 495333, ~: 496772)
[PASS] test_totalAssets((address[4],uint256[4],uint256[4],int256)) (runs: 5000, μ: 476539, ~: 478945)
[PASS] test_withdraw((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 5000, μ: 538903, ~: 540184)
[PASS] test_withdraw_zero_allowance((address[4],uint256[4],uint256[4],int256),uint256) (runs: 5000, μ: 498222, ~: 499903)
Suite result: ok. 27 passed; 0 failed; 0 skipped; finished in 80.05s (319.31s CPU time)
```

Ran 103 tests for test/unit/RewardRedistributor.t.sol:RewardRedistributorTest

```
[PASS] testCarryFairness() (gas: 1029467)
[PASS] testCarryMathematicalFormulas() (gas: 338495)
[PASS] testConservationAndSplit() (gas: 257806)
[PASS] testConservationInvariant() (gas: 238991)
[PASS] testConstructor_RevertsOnEOA_NotContract() (gas: 83965)
[PASS] testConstructor_RevertsOnInvalidUSDSC_ImplementsNeither() (gas: 146851)
[PASS] testConstructor_RevertsOnInvalidUSDSC_MissingIERC20() (gas: 184404)
[PASS] testConstructor_RevertsOnInvalidUSDSC_MissingIMYieldToOne() (gas: 801880)
[PASS] testConstructor_RevertsOnZeroUSDSCAddress() (gas: 79155)
[PASS] testDenominatorAndProportionality() (gas: 246755)
[PASS] testDonationBeforeDistribute() (gas: 317507)
[PASS] testDonationInvariants() (gas: 323294)
[PASS] testEarnVaultFundingOrder() (gas: 234493)
[PASS] testInvariant10_EventCorrectness() (gas: 265925)
[PASS] testInvariant1_ConservationOfValue() (gas: 241355)
[PASS] testInvariant2_CorrectDenominator() (gas: 68675)
[PASS] testInvariant2_PathologicalZeroSBase() (gas: 84520)
[PASS] testInvariant3_ProportionalAllocation() (gas: 67905)
[PASS] testInvariant4_LongRunFairness() (gas: 3118808)
[PASS] testInvariant5_OrderingAndFundingInvariants() (gas: 228484)
[PASS] testInvariant6_MonotonicNAV() (gas: 233404)
[PASS] testInvariant7_IdempotenceNoOp() (gas: 147575)
[PASS] testInvariant8_AccessControlAndPause() (gas: 145334)
[PASS] testInvariant9_EdgeCohorts() (gas: 166228)
[PASS] testMultipleDonations() (gas: 405202)
[PASS] testOrderingEarnVaultFundingInvariant() (gas: 228500)
[PASS] testPPSMonotonic() (gas: 230903)
[PASS] testPauseAndRoles() (gas: 140357)
[PASS] testPreviewDistributeConsistencyAcrossMultipleCalls() (gas: 79913)
[PASS] testPreviewDistributeConsistencyWithDonation() (gas: 318854)
[PASS] testPreviewDistributeEdgeCases() (gas: 86125)
[PASS] testPreviewDistributeSBaseCalculationInvariants() (gas: 245424)
[PASS] testPreviewDistributeWithCarryLogic() (gas: 241950)
```



```
[PASS] testRevert_DistributeWhenYieldRecipientChanged() (gas: 54689)
[PASS] testRevert_DistributeWhenYieldRecipientChangedMidOperation() (gas: 256222)
[PASS] testRevert_DistributeWhenYieldRecipientZero() (gas: 47780)
[PASS] testRoleManagement_ChangeOperatorAfterDeployment() (gas: 504216)
[PASS] testRoleRenunciation_AdminCanRenounceWhenMultipleAdminsExist() (gas: 102214)
[PASS] testRoleRenunciation_LastAdminCannotBeRenounced() (gas: 27034)
[PASS] testRoleRenunciation_NonAdminCanAttemptRenounceWithoutError() (gas: 36822)
[PASS] testRoleRenunciation_OperatorRoleCanBeRenounced() (gas: 56477)
[PASS] testRoleRevocation_CanRevokeAdminWhenMultipleExist() (gas: 92037)
[PASS] testRoleRevocation_CannotRevokeLastOfMultipleAdmins() (gas: 158824)
[PASS] testRoleRevocation_CannotRevokeSingleAdmin() (gas: 29909)
[PASS] testRoleRevocation_LastAdminProtectionWithRegrant() (gas: 143281)
[PASS] testRoleRevocation_MultiAdminScenario() (gas: 157316)
[PASS] testRoleRevocation_NonAdminCannotRevokeRoles() (gas: 38583)
[PASS] testRoleRevocation_NonAdminRevocationDoesNotTriggerProtection() (gas: 39487)
[PASS] testRoleRevocation_OperatorRoleCanBeRevoked() (gas: 63097)
[PASS] testZeroEligibleTVL_AllToStartale() (gas: 195907)
[PASS] test_DistributeSucceedsWhenYieldRecipientCorrect() (gas: 232690)
[PASS] test_DistributeWithZeroSBase() (gas: 225157)
[PASS] test_Distribute_RevertsWhenMaxAgeUpdatedAndSnapshotTooOld() (gas: 147651)
[PASS] test_Distribute_RevertsWhenNoSnapshotTaken() (gas: 50015)
[PASS] test_Distribute_RevertsWhenSameBlock() (gas: 132270)
[PASS] test_Distribute_RevertsWhenSnapshotTooOld() (gas: 137007)
[PASS] test_Distribute_SucceedsAfterMultipleBlocks() (gas: 232440)
[PASS] test_Distribute_SucceedsAfterNextBlock() (gas: 232399)
[PASS] test_Distribute_SucceedsAtMaxAgeBoundary() (gas: 231922)
[PASS] test_Distribute_SucceedsWhenSnapshotWithinValidRange() (gas: 230753)
[PASS] test_Distribute_UsesSnapshotTVLNotCurrentTVL() (gas: 291529)
[PASS] test_EdgeCase_MultipleSnapshotsBeforeDistribution() (gas: 242112)
[PASS] test_EdgeCase_SnapshotAtMaxTimestamp() (gas: 98288)
[PASS] test_EdgeCase_SnapshotAtZeroTimestamp() (gas: 84795)
[PASS] test_Errors_LastSnapshotInvalid_NoSnapshot() (gas: 56256)
[PASS] test_Errors_MustSnapshotInPreviousBlocks_SameBlock() (gas: 132249)
[PASS] test_Events_MultipleSnapshots_EmitsMultipleEvents() (gas: 109919)
[PASS] test_Events_SusdscTVLSnapshotCaptured_EmitsCorrectParameters() (gas: 108191)
[PASS] test_ExploitScenario_AttackerCannotFrontrunWithProperWorkflow() (gas: 295947)
[PASS] test_ExploitScenario_PreventsSameBlockTVLManipulation() (gas: 171650)
[PASS] test_ExploitScenario_ProperWorkflowPreventsExploit() (gas: 298211)
[PASS] test_ExploitScenario_RealisticAttackScenario() (gas: 325231)
[PASS] test_ExploitScenario_RequiresNewSnapshotAfterDistribution() (gas: 361481)
[PASS] test_PreviewDistribute_UsesSnapshotTVL() (gas: 177830)
[PASS] test_RecoverDonations_AfterTreasuryChange() (gas: 100407)
[PASS] test_RecoverDonations_BalanceInvariant_NoDonations() (gas: 232026)
[PASS] test_RecoverDonations_BalanceInvariant_WithDonations() (gas: 308788)
[PASS] test_RecoverDonations_BasicRecovery() (gas: 93687)
[PASS] test_RecoverDonations_DonationsAfterDistribute() (gas: 274780)
[PASS] test_RecoverDonations_DonationsNotDistributed() (gas: 323818)
[PASS] test_RecoverDonations_EventEmission() (gas: 81102)
[PASS] test_RecoverDonations_MultipleDistributionsWithDonations() (gas: 411078)
[PASS] test_RecoverDonations_MultipleDonations() (gas: 111282)
[PASS] test_RecoverDonations_NonReentrant() (gas: 122413)
[PASS] test_RecoverDonations_OnlyAdmin() (gas: 94937)
[PASS] test_RecoverDonations_ZeroBalance() (gas: 40067)
[PASS] test_SetSnapshotMaxAge_CanSetToMaximum() (gas: 22097)
[PASS] test_SetSnapshotMaxAge_CanSetToMiddleValue() (gas: 22095)
[PASS] test_SetSnapshotMaxAge_CanSetToMinimum() (gas: 22117)
[PASS] test_SetSnapshotMaxAge_DefaultValue() (gas: 11454)
[PASS] test_SetSnapshotMaxAge_EmitsEvent() (gas: 25659)
[PASS] test_SetSnapshotMaxAge_OnlyAdmin() (gas: 14315)
[PASS] test_SetSnapshotMaxAge_RevertsWhenLessThanMinimum() (gas: 15488)
[PASS] test_SetSnapshotMaxAge_RevertsWhenMoreThan7Days() (gas: 15511)
[PASS] test_SnapshotSusdscTVL_CapturesBlockNumberCorrectly() (gas: 109373)
[PASS] test_SnapshotSusdscTVL_CapturesCorrectTVL() (gas: 107245)
[PASS] test_SnapshotSusdscTVL_CapturesLargeTVL() (gas: 117234)
[PASS] test_SnapshotSusdscTVL_CapturesZeroTVL() (gas: 96461)
[PASS] test_SnapshotSusdscTVL_EmitsEvent() (gas: 102504)
[PASS] test_SnapshotSusdscTVL_InitialStateIsZero() (gas: 15568)
[PASS] test_SnapshotSusdscTVL_OnlyOperator() (gas: 13741)
[PASS] test_SnapshotSusdscTVL_RevertsWhenPaused() (gas: 26505)
[PASS] test_SnapshotSusdscTVL_UpdatesOnMultipleCalls() (gas: 126060)
Suite result: ok. 103 passed; 0 failed; 0 skipped; finished in 80.05s (157.06ms CPU time)
```



```
Ran 16 tests for test/invariants/SUSDSCVaultInvariants.t.sol:SUSDSCVaultInvariants
[PASS] invariant_assetConservation() (runs: 512, calls: 25600, reverts: 0)
```

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

[illegible]

```
Bound result 13
Bound result 9
Bound result 74104302177635202094361
Bound result 16
Bound result 8083010326599365693462
Bound result 1
Bound result 23287809584608390126301
Bound result 7
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 3489811130201797597125
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25
```

```
[PASS] invariant_assetIntegrity() (runs: 512, calls: 25600, reverts: 0)
```

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

[illegible]

Bound result 18
Bound result 6
Bound result 999900000000000021815
Bound result 7
Bound result 15
Bound result 86062875891763093953281
Bound result 14
Bound result 589192616948811
Bound result 7029603753124642334572
Bound result 1
Bound result 74104302177635202094361
Bound result 5
Bound result 999999999999999999997
Bound result 0
Bound result 58703158656766531479830
Bound result 8
Bound result 11
Bound result 12317793141521667
Bound result 12
Bound result 5259
Bound result 9999511332147241617563
Bound result 13
Bound result 2924
Bound result 13
Bound result 74104302177635202094361
Bound result 16
Bound result 8083010326599365693462
Bound result 1
Bound result 47248160047008619131804
Bound result 7
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 19907064597569720658863
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25

[PASS] invariant_conversionConsistency() (runs: 512, calls: 25600, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

Bound result 14
Bound result 4348821872561024
Bound result 1
Bound result 4
Bound result 74104302177635202094361
Bound result 10
Bound result 0
Bound result 38446643297351465963616
Bound result 4265358736228299581939
Bound result 9
Bound result 8


```
Bound result 22206293210690767957345
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25
```

```
[PASS] invariant_maxFunctionsNeverRevert() (runs: 512, calls: 25600, reverts: 0)
```

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

[illegible]

Bound result 1
Bound result 74104302177635202094361
Bound result 5
Bound result 999999999999999999997
Bound result 0
Bound result 58703158656766531479830
Bound result 8
Bound result 5
Bound result 80177657135192497079793
Bound result 0
Bound result 4855
Bound result 9999511332147241617563
Bound result 13
Bound result 11
Bound result 74104302177635202094361
Bound result 16
Bound result 8083010326599365693462
Bound result 1
Bound result 3079306869334812252651
Bound result 7
Bound result 103236808852681186
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 2306720979625939460198
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25

[PASS] invariant_noArbitraryMinting() (runs: 512, calls: 25600, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

Bound result 14
Bound result 4348821872561024
Bound result 6
Bound result 9
Bound result 74104302177635202094361
Bound result 10
Bound result 0
Bound result 38446643297351465963616
Bound result 4265358736228299581939
Bound result 8
Bound result 8
Bound result 9
Bound result 999999999999999999997
Bound result 11
Bound result 147196117405101571
Bound result 18
Bound result 8261
Bound result 13
Bound result 1495665375853932236767
Bound result 12

Bound result 18
Bound result 1
Bound result 2
Bound result 4
Bound result 8043710824041591340642
Bound result 12
Bound result 1
Bound result 1
Bound result 4146090862159595981733
Bound result 8
Bound result 16
Bound result 12
Bound result 40601917066085016906649
Bound result 18
Bound result 6
Bound result 999900000000000021804
Bound result 7
Bound result 15
Bound result 86062875891763093953281
Bound result 14
Bound result 589192616948811
Bound result 7029603753124642334572
Bound result 1
Bound result 74104302177635202094361
Bound result 5
Bound result 9999999999999999999999997
Bound result 0
Bound result 58703158656766531479830
Bound result 8
Bound result 16
Bound result 18
Bound result 5257
Bound result 9999511332147241617563
Bound result 13
Bound result 14
Bound result 74104302177635202094361
Bound result 16
Bound result 8083010326599365693462
Bound result 1
Bound result 5333883611845886182585
Bound result 7
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 22206293210690767957345
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25

[PASS] invariant_noNegativeBalances() (runs: 512, calls: 25600, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

Bound result 16
Bound result 12
Bound result 40601917066085016906649
Bound result 18
Bound result 6
Bound result 999900000000000021931
Bound result 7
Bound result 15
Bound result 86062875891763093953281
Bound result 14
Bound result 4598536235780127
Bound result 7029603753124642334572
Bound result 1
Bound result 74104302177635202094361
Bound result 5
Bound result 999999999999999999997
Bound result 0
Bound result 58703158656766531479830
Bound result 8
Bound result 14
Bound result 35092101822729
Bound result 6
Bound result 4754
Bound result 9999511332147241617563
Bound result 13
Bound result 6
Bound result 74104302177635202094361
Bound result 16
Bound result 8083010326599365693462
Bound result 1
Bound result 53888594050442881081433
Bound result 7
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 1925062480397915605341
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25

[PASS] invariant_previewNeverReverts() (runs: 512, calls: 25600, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

Bound result 14
Bound result 4348821872561024
Bound result 1
Bound result 4
Bound result 74104302177635202094361
Bound result 10
Bound result 0
Bound result 38446643297351465963616
Bound result 4265358736228299581939


```
Bound result 5514744683811900808941
Bound result 7
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 63452667599427393058393
Bound result 17
Bound result 64092584269511547841
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25
```

[PASS] invariant_rolesSecurity() (runs: 512, calls: 25600, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

[illegible]

Bound result 7
Bound result 15
Bound result 86062875891763093953281
Bound result 14
Bound result 977960150270417
Bound result 7029603753124642334572
Bound result 1
Bound result 74104302177635202094361
Bound result 5
Bound result 999999999999999999999999999999997
Bound result 0
Bound result 58703158656766531479830
Bound result 8
Bound result 10577510832648450993742
Bound result 12
Bound result 30349385475987408228267
Bound result 15
Bound result 4833
Bound result 9999511332147241617563
Bound result 13
Bound result 15
Bound result 74104302177635202094361
Bound result 16
Bound result 8083010326599365693462
Bound result 1
Bound result 5619631894458262047833
Bound result 7
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 28515269864143902137697
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25

[PASS] invariant_roundingFavorVault() (runs: 512, calls: 25600, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0
SUSDSCVaultHandler	mint	5208	0	0
SUSDSCVaultHandler	redeem	5179	0	0
SUSDSCVaultHandler	withdraw	5150	0	0

Logs:

Bound result 14
Bound result 4348821872561024
Bound result 1
Bound result 4
Bound result 74104302177635202094361
Bound result 10
Bound result 0
Bound result 38446643297351465963616
Bound result 4265358736228299581939
Bound result 9
Bound result 8
Bound result 9
Bound result 999999999999999999999999999999997
Bound result 11

Bound result 147196117405101571
Bound result 13
Bound result 8273
Bound result 13
Bound result 2690
Bound result 1495665375853932236767
Bound result 12
Bound result 18
Bound result 2
Bound result 4
Bound result 8043710824041591340642
Bound result 12
Bound result 1
Bound result 1
Bound result 4146090862159595981733
Bound result 8
Bound result 16
Bound result 12
Bound result 40601917066085016906649
Bound result 18
Bound result 6
Bound result 999900000000000021817
Bound result 7
Bound result 15
Bound result 86062875891763093953281
Bound result 14
Bound result 589192616948811
Bound result 7029603753124642334572
Bound result 1
Bound result 74104302177635202094361
Bound result 5
Bound result 999999999999999999997
Bound result 0
Bound result 58703158656766531479830
Bound result 8
Bound result 12
Bound result 19315212425447608507070
Bound result 9
Bound result 5259
Bound result 9999511332147241617563
Bound result 13
Bound result 2924
Bound result 11
Bound result 74104302177635202094361
Bound result 16
Bound result 8083010326599365693462
Bound result 1
Bound result 64787263996255286932777
Bound result 7
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 837082567417451896071
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25

[PASS] invariant_shareSupplyConsistency() (runs: 512, calls: 25600, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
SUSDSCVaultHandler	addYield	5007	0	0
SUSDSCVaultHandler	deposit	5056	0	0

Bound result 4
Bound result 8043710824041591340642
Bound result 12
Bound result 1
Bound result 1
Bound result 4146090862159595981733
Bound result 8
Bound result 16
Bound result 12
Bound result 40601917066085016906649
Bound result 18
Bound result 6
Bound result 999900000000000021804
Bound result 7
Bound result 15
Bound result 86062875891763093953281
Bound result 14
Bound result 2089180575946447
Bound result 7029603753124642334572
Bound result 1
Bound result 74104302177635202094361
Bound result 5
Bound result 9999999999999999999999997
Bound result 0
Bound result 58703158656766531479830
Bound result 8
Bound result 17
Bound result 19
Bound result 5258
Bound result 9999511332147241617563
Bound result 13
Bound result 5
Bound result 74104302177635202094361
Bound result 16
Bound result 8083010326599365693462
Bound result 1
Bound result 26766577399552313515383
Bound result 7
Bound result 4
Bound result 89098802570810369274226
Bound result 1
Bound result 40838018951707617730673
Bound result 17
Bound result 0
Bound result 64967423020029457339374
Bound result 2
Bound result 52766292615423543946909
Bound result 1
Bound result 74104302177635202094361
Bound result 16
Bound result 25

Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 80.05s (433.56s CPU time)

Ran 15 test suites in 80.20s (420.95s CPU time): 544 tests passed, 0 failed, 0 skipped (544 total tests)

Code Coverage

Coverage data was obtained using `forge coverage --no-match-coverage '(script|test|mock|coin|4626)'`.

Update: Branch coverage has increased for `RewardRedistributor`, although it has decreased for the other contracts in scope.

File	% Lines	% Statements	% Branches	% Funcs
src/distributor/RewardRedistri butor.sol	96.55% (140/145)	93.17% (150/161)	82.05% (32/39)	100.00% (20/20)

File	% Lines	% Statements	% Branches	% Funcs
src/vaults/earn/BoostRewardsLib.sol	73.85% (48/65)	75.68% (56/74)	56.52% (13/23)	100.00% (4/4)
src/vaults/earn/EarnVault.sol	97.58% (242/248)	91.64% (252/275)	61.40% (35/57)	100.00% (40/40)
src/vaults/earn/EarnVaultStorageBase.sol	100.00% (4/4)	100.00% (3/3)	100.00% (0/0)	100.00% (2/2)
src/vaults/earn/EarnVaultUpgradeable.sol	92.14% (293/318)	88.24% (330/374)	63.16% (36/57)	96.15% (50/52)
Total	93.21% (727/780)	89.18% (791/887)	65.91% (116/176)	98.31% (116/118)

Changelog

- 2025-11-07 - Initial report
- 2025-11-20 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp’s mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp’s collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

